

```

353     irq_desc[i].action = 0;
354     irq_desc[i].depth = 1;
355
356     if (i < 16) {
357         /*
358          * 16 old-style INTA-cycle interrupts:
359          */
360         irq_desc[i].handler = &i8259A_irq_type;
361     } else {
362         /*
363          * 'high' PCI IRQs filled in on demand
364          */
365         irq_desc[i].handler = &no_irq_type;
366     }
367 }
368 }
...
409
410 void __init init_IRQ(void)
411 {
412     int i;
413
414     /* all the set up before the call gates are initialized */
415     pre_intr_init_hook();
...
422     for (i = 0; i < NR_IRQS; i++) {
423         int vector = FIRST_EXTERNAL_VECTOR + i;
424         if (vector != SYSCALL_VECTOR)
425             set_intr_gate(vector, interrupt[i]);
426     }
...
431     intr_init_hook();
...
437     setup_timer();
...
}
-----

```

第 410 行：这是从 `start_kernel()` 调用的函数入口点，而 `start_kernel()` 是在系统启动时调用的主要的内核初始化函数。

第 342~348 行：如果本地 APIC 是可用的，并的确需要使用，那就将它初始化并设置为虚导线模式，用来和 8259 配合工作。然后，利用 `init_8259A(0)` 中的 I/O 寄存器来初始化 8259 设备。

第 422~426 行：在第 424 行，这个循环没有对中断向量 `syscalls` 进行设置，因为它早在 `tRap_init()` 中已经设置过了。Linux 使用 Intel 的中断门（已由内核代码初始化）作为中断描述符。这是由 `set_intr_gate()` 宏（在第 425 行）来设置的。异常则使用 Intel 的系统门和陷阱门，分别由 `set_system_gate()` 和 `set_trap_gate()` 来设置。这些宏是在文件 `arch/i386/kernel/traps.c` 中定义的。

第 431 行：为本地 APIC（如果使用的话）设置中断处理程序，并且为级联的 8259 调用 `irq.c` 中的 `setup_irq()`。

第 437 行：用 I/O 寄存器启动 8253 PIT。

第 419 行^①：在 `init/main.c` 代码中的第 419 行，遵照 `time_init()` 为 PPC 和 x86 安装系统时钟中断处理程序的函数。在 PPC 中，系统定时器初始化衰减器（为简单起见进行了缩简）：

```
-----
arch/ppc/kernel/time.c
void __init time_init(void)
{
...
317   ppc_md.calibrate_decr();
...
351   set_dec(tb_ticks_per_jiffy);
...
}
```

第 317 行：为系统的 HZ 值给出合适的数值。

第 351 行：为衰减器设置合适的初始值。

PowerPC 的中断体系结构及其在 Linux 中的实现不要求安装时钟中断。衰减器中断向量位于 0x900 处。处理程序的调用被硬编码在此处，并且不被共享。

```
-----
arch/ppc/kernel/head.S
/* Decrementer */
479  EXCEPTION(0x900, Decrementer, timer_interrupt, EXC_XFER_LITE)
-----
```

本节稍后将详细介绍操作衰减器的 `EXCEPTION` 宏。当达到终止计数时，衰减器的处理程序就做好被执行的准备。

下列代码片断简要描述 x86 系统定时器的初始化。

```
-----
arch/i386/kernel/time.c
void __init time_init(void)
{
...
340   time_init_hook();
}
```

函数 `time_init()` 调用 `time_init_hook()`，后者位于和机器相关的设置文件 `setup.c` 中。

```
-----
arch/i386/machine-default/setup.c
072 static struct irqaction irq0 = { timer_interrupt, SA_INTERRUPT, 0, "timer", NULL, NULL};
...
081 void __init time_init_hook(void)
082 {
083   setup_irq(0, &irq0);
084 }
```

① 这一行代码和上面所列的代码也不是同一个文件，在 `init/main.c` 代码中调用，在 `arch/i386/kernel/time.c` 中实现。

第 72 行：初始化与 irq0 对应的 irqaction 结构体。

第 81~84 行：该函数调用 setup_irq(0, &irq0)，将包含处理程序 timer_interrupt() 的 irqaction 结构体放到与 irq0 相关的共享中断队列上。

对于一般的处理程序而言（那些不在内核初始化时装载的处理程序），这段代码与调用 request_irq() 的效果类似。时钟中断的初始化代码通过一个捷径把处理程序放入 irq_desc[] 中。运行时代码使用 irq.c 中的 disable_irq()、enable_irq()、request_irq() 和 free_irq()。所有这些函数都是能对 IRQ 起作用的实用程序，并在某一处涉及 irq_desc 结构。

● 中断时机

对于 PowerPC 而言，衰减器在处理器的内部，并让其中断向量位于地址 0x900 处。这和 x86 体系结构恰恰相反，在 x86 体系结构中，PIT 是一个来自于中断控制器的外部中断。PowerPC 的外部中断控制器位于向量 0x500 处。如果系统定时器是基于本地 APIC 的，那么 x86 体系结构中的情形也就和 PowerPC 中的情形类似了。

表 3-10 和表 3-11 分别描述 x86 和 PPC 体系结构的中断向量表。

表3-10 x86的中断向量表

中断向量号/IRQ	说 明
0	除法错误
1	调试扩展
2	NMI（不可屏蔽的）中断
3	断点
4	INTO指令检测到的溢出
5	BOUND检测到的越界
6	无效的操作码
7	设备不可用
8	检测到两个错误
9	协处理器段越界（保留）
10	无效的任务状态段
11	段不存在
12	栈错误
13	通用保护
14	缺页
15	（Intel预留，不要使用）
16	浮点错误
17	对齐检查
18	机器校验
19~31	（Intel预留，不要使用）
32~255	可屏蔽中断

表3-11 PPC中的中断向量偏移

偏移量 (十六进制)	中断类型
00000	保留
00100	系统复位
00200	机器校验
00300	数据存储
00400	指令存储
00500	外部中断
00600	对齐
00700	程序
00800	浮点不可用
00900	衰减器
00A00	保留
00B00	保留
00C00	系统调用
00D00	跟踪
00E00	浮点辅助
00E10	保留
...	...
00FFF	保留
01000	保留, 取决于具体实现
...	...
02FFF	(中断向量区域结束)

注意两种体系结构间的相似之处。这两个表描述的都是硬件。Intel 异常中断向量表的软件接口就是本章前面所提到的中断描述符表 (IDT)。

继续深入分析, 就会看到 Intel 体系结构处理硬件中断的全过程, 首先将 IRQ 传给处理器, 然后转到 entry.S 中的跳转表, 找到相应的调用门 (描述符), 最后获得处理程序的代码。图 3-19 说明了这个过程。

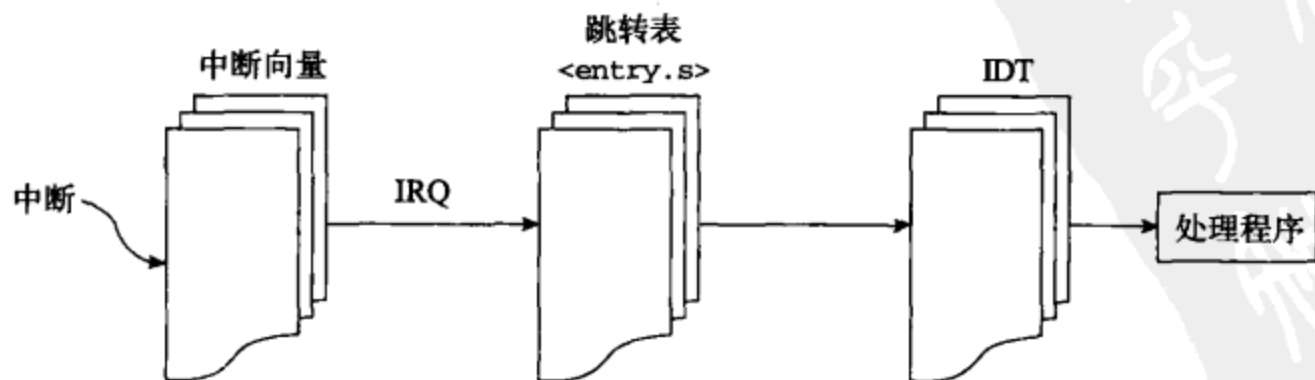


图 3-19 x86 的中断处理过程

另一方面, PowerPC 指向内存中明确的偏移处, 此处放置的是跳转到合适的中断处理程序的

代码。从图 3-20 中可以看到，位于 head.S 中的 PPC 跳转表通过在内存中被固定的方式被索引。图 3-20 展示了这一过程。

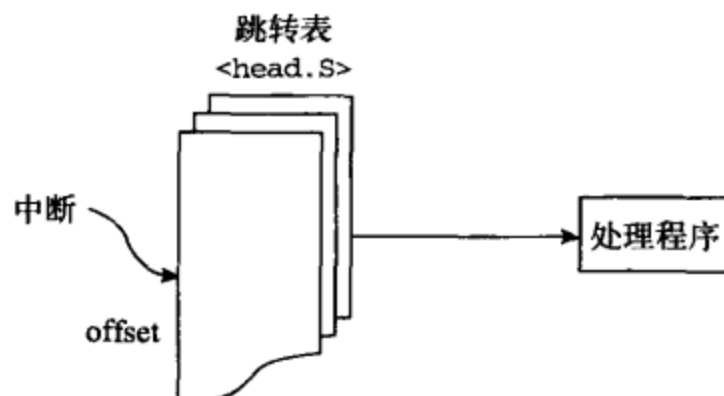


图 3-20 PPC 的中断处理过程

分析 PPC 的外部中断（偏移量 0x500）处理程序和定时器中断（偏移量 0x900）处理程序时，PPC 的中断处理过程会更清楚一些。

● 处理 PowerPC 外部中断向量

与前面所讨论的一样，如果外部中断发生，处理器就跳转到地址 0x500 处。进一步深入分析文件 head.S 中的 EXCEPTION() 宏后，可以看到下面几行代码被链接并装载，以便它被映射到偏移地址为 0x500 的这片内存区域。这种结构化的跳转表和 x86 的 IDT 作用相同：

```
-----
arch/ppc/kernel/head.S
453  /* External interrupt */
454  EXCEPTION(0x500, HardwareInterrupt, do_IRQ, EXC_XFER_LITE)

The third parameter, do_IRQ(), is called next. Let's take a look at this function.
arch/ppc/kernel/irq.c
510  void do_IRQ(struct pt_regs *regs)
511  {
512  int irq, first = 1;
513  irq_enter();
...
523  while ((irq = ppc_md.get_irq(regs)) >= 0) {
524  ppc_irq_dispatch_handler(regs, irq);
525  first = 0;
526  }
527  if (irq != -2 && first)
528  /* That's not SMP safe ... but who cares ? */
529  ppc_spurious_interrupts++;
530  irq_exit();
531  }
-----
```

第 513~530 行：表示正处于硬件中断中的抢占代码。

第 523 行：从中断控制器读取一个待处理的中断，并转换成 IRQ 号（直至所有的中断都被处理）。

第 524 行：ppc_irq_dispatch_handler() 处理中断。下面将更详细地分析这个函数。

函数 `ppc_irq_dispatch_handler()` 几乎等同于 x86 的函数 `do_IRQ()`。

```
-----
arch/ppc/kernel/irq.c
428 void ppc_irq_dispatch_handler(struct pt_regs *regs, int irq)
429 {
430     int status;
431     struct irqaction *action;
432     irq_desc_t *desc = irq_desc + irq;
433
434     kstat_this_cpu.irqs[irq]++;
435     spin_lock(&desc->lock);
436     ack_irq(irq);
437
438     ...
441     status = desc->status & ~(IRQ_REPLAY | IRQ_WAITING);
442     if (!(status & IRQ_PER_CPU))
443         status |= IRQ_PENDING; /* we _want_ to handle it */
444
445     ...
449     action = NULL;
450     if (likely(!(status & (IRQ_DISABLED | IRQ_INPROGRESS)))) {
451         action = desc->action;
452         if (!action || !action->handler) {
453             ppc_spurious_interrupts++;
454             printk(KERN_DEBUG "Unhandled interrupt %x, disabled\n", irq);
455             /* We can't call disable_irq here, it would deadlock */
456             ++desc->depth;
457             desc->status |= IRQ_DISABLED;
458             mask_irq(irq);
459             /* This is a real interrupt, we have to eoi it,
460              so we jump to out */
461             goto out;
462         }
463         status &= ~IRQ_PENDING; /* we commit to handling */
464         if (!(status & IRQ_PER_CPU))
465             status |= IRQ_INPROGRESS; /* we are handling it */
466     }
467     desc->status = status;
468
469     ...
489     for (;;) {
490         spin_unlock(&desc->lock);
491         handle_irq_event(irq, regs, action);
492         spin_lock(&desc->lock);
493
494         if (likely(!(desc->status & IRQ_PENDING)))
495             break;
496         desc->status &= ~IRQ_PENDING;
497     }
498 out:
499     desc->status &= ~IRQ_INPROGRESS;
500
501     ...
511 }
```

第 432 行：从参数获得 IRQ 并获取对适当 `irq_desc` 的访问。

第 435 行：获得 IRQ 描述符上的自旋锁，以防同一个中断被不同的 CPU 并发访问。

第 436 行：给硬件发送一个确认，随后，硬件给出回应，在这个中断处理完之前，防止更多该类型的中断被处理。

第 441~443 行：标志 IRQ_REPLAY 和 IRQ_WAITING 被清零。在这种情况下，IRQ_REPLAY 表示该 IRQ 曾经被丢弃，现在准备重新发送，IRQ_WAITING 表示 IRQ 正在被测试。（这两种情况都超出了本书的讨论范围。）在单处理器系统中，设置 IRQ_PENDING 标志，表示答应负责处理中断。

第 450 行：这段代码检查不处理中断的情形。如果设置了 IRQ_DISABLED 或 IRQ_INPROGRESS 就可以忽略这段代码。当不想让系统响应正在处理的特定 IRQ 线时，就设置 IRQ_DISABLED。IRQ_INPROGRESS 表示中断正在被一个处理器处理，这用在多处理器系统的第二个处理器试图产生同一个中断的情况。

第 451~462 行：此处查看中断处理程序是否存在。如果不存在，就跳转到第 498 行的标号“out”处。

第 463~465 行：此处将三个不处理中断的条件全部清除，因此允许处理中断，即设置 IRQ_INPROGRESS 标志，清除 IRQ_PENDING 标志，表示中断正在处理。

第 489~497 行：中断得到处理，在处理中断之前，释放中断描述符上的自旋锁。自旋锁被释放后，将调用 handle_irq_event() 例程。这个例程将执行中断处理程序。执行完毕后就会再次获得描述符上的自旋锁。如果在 IRQ 处理期间没有其他 CPU 设置 IRQ_PENDING 标志，就跳出循环。否则，再次处理中断。

● 处理 PowerPC 的系统时钟中断

在 timer_init() 中已经指出，衰减器被硬编码到地址 0x900 处。假定已经达到终止计数，此时将调用 arch/ppc/kernel/time.c 中的处理程序 timer_interrupt()。

```
-----
arch/ppc/kernel/head.S
/* Decrementer */
479 EXCEPTION(0x900, Decrementer, timer_interrupt, EXC_XFER_LITE)
-----
```

这是 timer_interrupt() 函数。

```
-----
arch/ppc/kernel/time.c
145 void timer_interrupt(struct pt_regs * regs)
146 {
...
152 if (atomic_read(&ppc_n_lost_interrupts) != 0)
153     do_IRQ(regs);
154
155 irq_enter();
...
159 if (!user_mode(regs))
160     ppc_do_profile(instruction_pointer(regs));
...
165 write_seqlock(&xtime_lock);
```

```

166
167     do_timer(regs);
...
189     if (ppc_md.set_rtc_time(xtime.tv_sec+1 + time_offset) == 0)
...
195     write_sequnlock(&xtime_lock);
...
198     set_dec(next_dec);
...
208     irq_exit();
209 }

```

第 152 行：如果中断丢失，则回去调用 0x900 处的外部中断处理程序。

第 159 行：对内核路径调试进行内核分析。

第 165 和 195 行：把这段代码锁在外面。

第 167 行：这行代码和 x86 时钟中断使用的是同一个函数（下面将讨论）。

第 189 行：更新 RTC。

第 198 行：为下一个中断重新启动衰减器。

第 208 行：从中断返回。被中断的代码现在正常运行，直到下一个中断出现。

● 处理 x86 的系统时钟中断

中断激活时（在本例中，PIT 倒计时到 0 并激活 IRQ0），中断控制器激活连入处理器的中断请求线。entry.S 中的汇编代码有对应于 IDT 中每个描述符的入口点。IRQ0 是第一个外部中断，在 IDT 中是向量 32。于是，这段代码准备跳转到位于 entry.S 中的跳转表的入口点 32 处。

```

-----
arch/i386/kernel/entry.S
385     vector=0
386     ENTRY(irq_entries_start)
387     .rept NR_IRQS
388     ALIGN
389     1:     pushl $vector-256
390     jmp common_interrupt
391     .data
392     .long 1b
393     .text
394     vector=vector+1
395     .endr
396
397     ALIGN
398     common_interrupt:
399     SAVE_ALL
400     call do_IRQ
401     jmp ret_from_intr
-----

```

这是一段精彩的汇编程序。重复结构 .rept（在第 387 行）以及结束语句（在第 395 行）在编译时创建中断跳转表。注意，当这个代码块重复创建时，第 389 行压入栈的向量号是递减的。把向量压入栈，内核代码就知道，中断发生时它正在对哪一个 IRQ 进行处理。

当离开对 x86 代码的跟踪时，代码就跳转到跳转表中适当的入口点，并把 IRQ 存放到栈中，然后跳转到第 398 行的公用处理程序，并调用第 400 行的 `do_IRQ()` (`arch/i386/kernel/irq.c`)。这个函数几乎和 `ppc_irq_dispatch_handler()` 完全相同，`ppc_irq_dispatch_handler()` 在本节的“处理 PowerPC 外部中断向量”中已经描述过，此处不再赘述。

函数 `do_irq()` 可以根据传入的 IRQ 来访问 `irq_desc` 中的适当元素，并跳转到 `action` 结构链表中的每个中断处理程序。此处，实际上是让它成为 PIT 的处理函数：`timer_interrupt()`，参见 `time.c` 中的下列代码段。处理程序从第 274 行处开始，并保持和源文件中相同的次序：

```
-----
arch/i386/kernel/time.c
274 irqreturn_t timer_interrupt(int irq, void *dev_id, struct pt_regs *regs)
275 {
...
287 do_timer_interrupt(irq, NULL, regs);
...
290 return IRQ_HANDLED;
291 }
-----
```

第 274 行：这是系统时钟中断处理程序的入口点。

第 287 行：这是调用函数 `do_timer_interrupt()`。

```
-----
arch/i386/kernel/time.c
208 static inline void do_timer_interrupt(int irq, void *dev_id,
209 struct pt_regs *regs)
210 {
...
227 do_timer_interrupt_hook(regs);
...
250 }
-----
```

第 227 行：调用函数 `do_timer_interrupt_hook()`。这个函数本质上是对 `do_timer()` 调用的封装。让我们来看看：

```
-----
include/asm-i386/mach-default/do_timer.h
016 static inline void do_timer_interrupt_hook(struct pt_regs *regs)
017 {
018 do_timer(regs);
...
025 x86_do_profile(regs);
...
030 }
-----
```

第 18 行：调用函数 `do_timer()`。这个函数完成更新系统时间的大部分工作。

第 25 行：`x86_do_profile()` 程序用于查看 `eip` 寄存器，`eip` 寄存器存放中断前正在运行的代码。这个数据表示，随着时间的推移，进程运行的频繁程度。

在这里，系统时钟中断从 `do_irq()` 返回到 `entry.S` 进行系统事务管理，被中断的线程恢复执行。

如前所述，系统定时器是 Linux 操作系统的脉搏。尽管本章中定时器作为中断的一个实例，实际上它的用途遍及整个操作系统。

3.9 小结

进程必须和其他进程共享处理器并定义各自的执行上下文，执行上下文包含进程运行所必须的所有信息。在它们的执行期间经历了各种状态，这些状态可被抽象成阻塞状态、运行状态和就绪状态。

内核把有关进程的信息存放在 `task_struct` 描述符中，根据进程所具有的不同功能，可以把 `task_struct` 的字段划分为：进程的属性、进程间的关系、进程的内存访问、与进程相关的文件操作、进程的信任状、资源限制以及调度。所有这些字段都是记录进程上下文所必须的。进程可由一个或多个共享内存地址空间的线程组成。每个线程都有自己的结构。

通过调用 `fork()`、`vfork()` 或 `clone()` 中的某一个系统调用，一个新的进程就此诞生。这三个系统调用最终都调用了内核例程 `do_fork()`，`do_fork()` 完成新进程创建的大部分工作。在执行期间，进程从一个状态转换到另一个状态。进程经由调度程序的选择从就绪状态转换到运行状态；如果它的时间片用完或者如果它把 CPU 让给其他进程，就会从运行状态转换到就绪状态；如果等待的信号到来，就从阻塞状态转换到就绪状态；当进程等待一个资源或进程睡眠时，就从运行状态转换到阻塞状态。进程的消亡发生在调用 `exit()` 时。

本章还深入研究调度程序的基本架构以及它所使用的数据结构，包括运行队列、等待队列以及怎样管理这些结构以便记录进程被调度的踪迹。

通过研究 x86 和 PPC 的硬件如何处理中断，本章最后讨论了进程的异步执行流程（包括异常和中断）。我们以系统定时器为例，分析了 Linux 内核如何管理硬件提交的中断。

3.9.1 项目：系统变量 `current`

本章分析了 `task_struct` 和 `current`。`current` 是系统变量，它指向当前执行进程的 `task_struct`。本项目的目标是加强这样一个概念，即内核是一系列有序而不断变化的连环结构，随着程序运行这些结构被创建和销毁。如我们所见，`task_struct` 结构是内核所拥有的最重要的结构之一，其中含有给定进程所必须的全部信息。这个项目模块如同内核那样访问 `task_struct` 结构，并为读者今后的进一步研究打下基础。

在本项目中，将访问当前进程的 `task_struct` 并输出该结构的各种元素。从文件 `include/linux/sched.h` 出发，直到找到 `task_struct`。可以借助文件 `sched.h` 来引用 `current->pid`、`current->comm` 和当前进程的 ID 及其名字，并沿着这些结构找到父进程的 `pid` 和 `comm`。接下来，借用 `printk()` 的一个例程为正在使用的 tty 终端回送一个消息。

请看下列代码。

注意 从运行第一个程序 (`hellomod`) 开始，当初始化例程打印出 `current->comm` 时，当前进程的名字会是什么呢？父进程的名字又会是什么呢？请看下列代码的分析。

3.9.2 项目源码^①

```

-----
currentptr.c
001  #include <linux/module.h>
002  #include <linux//kernel.h>
003  #include <linux/init.h>
004  #include <linux/sched.h>
005  #include <linux/tty.h>
006
007      void tty_write_message1(struct tty_struct *, char *);
008
009      static int my_init( void )
010      {
011
012          char *msg="Hello tty!";
013
014          printk("Hello, from the kernel...\n");
015          printk("parent pid =%d(%s)\n",current->parent->pid,current->
parent->comm);
016          printk("current pid =%d(%s)\n",current->pid,current->comm);
017
018          tty_write_message1(current->signal->tty,msg);
019          return 0;
020      }

022  static void my_cleanup( void )
{
    printk("Goodbye, from the kernel...\n");
}

027  module_init(my_init);
028  module_exit(my_cleanup);

// This routine was borrowed from <printk.c>
032  void tty_write_message1(struct tty_struct *tty, char *msg)
{
    if (tty && tty->driver->write)
        tty->driver->write(tty, 0, msg, strlen(msg));
    return;
037  }
-----

```

第 4 行: sched.h 包含 struct task_struct{}, 我们从这里引用进程的 ID (->pid) 和当前进程的名字(->comm), 以及指向父进程 PID (->parent) 的指针, 该指针指向父进程的 task 结构。同样也会找到指向信号结构的指针, 信号结构包含对 tty 结构的引用 (参见第 18~22 行)。

第 5 行: tty.h 包含 struct tty_struct{}, 我们从 printk.c 借用的例程要使用它 (见第 32~37 行)。

第 12 行: 这是将回送给终端的简单消息字符串。

第 15 行: 此处从当前进程的 task 结构引用其父进程的 PID 及其名字。前面问题的答案是:

^① 你可以把项目源码作为探究正在运行的内核的起点。尽管可以使用内核中许多有用的例程来查看信息, 如内部函数 (例如, strace()), 但构建如本项目这样的自定义工具, 可以展现出 Linux 内核实时性的一面。

该进程的父进程是当前的 shell 程序，在本例中，它就是 Bash。

第 16 行：此处从当前进程的 task 结构引用它的 PID 及其名字。为了回答前面问题的前半部分，只要在 Bash 命令行上输入 `insmod`，它就会被作为当前进程打印出来。

第 18 行：这是从 `kernel/printk.c` 借用的函数。它用于把消息重定向到指定的 tty。为了说明 `current` 的要点，可以从调用我们程序的地方把 tty（窗口或命令行）的 `tty_struct` 传递给这个例程，这是通过 `current->signal->tty` 来引用的。参数 `msg` 是在第 12 行声明的字符串。

第 32~38 行：`tty write` 函数检查到 tty 的存在，接着，以消息为参数调用适当的设备驱动程序。

3.9.3 运行代码

编译，并像第一个项目那样 `insmod()` 代码。

3.10 习题

1. 描述进程状态时，我们把“等待或阻塞”状态描述为进程没有运行也没有就绪时所处的状态。等待和阻塞之间有什么区别？在什么条件下进程处于等待状态，什么条件下它处于阻塞状态？

2. 找出把进程从运行状态设置为阻塞状态的内核代码。换句话说，找出 `current->state` 的状态从 `TASK_RUNNING` 转换到 `TASK_STOPPED` 的地方。

3. 为了弄清楚计数器的“倒计时”花费多少时间，请完成下面的计算。如果 64 位的衰减器以 500MHz 运行，终止下列值要用多长时间？

- a. `0x000000000000ffff`
- b. `0x00000000ffffffff`
- c. `0xffffffffffffff`

4. 当某段代码不应该被中断时，老版本的 Linux 是利用 `sti()` 和 `cli()` 来禁用中断的，而新版本的 Linux 使用 `spin_lock()`。自旋锁的主要优势是什么？

5. x86 的例程 `do_IRQ()` 和 PPC 的例程 `ppc_irq_dispatch_handler()` 如何处理共享的中断？

6. 为什么不推荐由内核代码访问系统调用？

7. 在运行 2.6 内核的 Linux 系统中，每个 CPU 有几个运行队列？

8. 当进程创建新进程时，Linux 要求它放弃一部分时间片吗？如果是，为什么？

9. 在进程的时间片到期后，怎样把它重新插入运行队列的优先级数组中？进程优先级的正常范围是多少？实时进程的优先级范围呢？

第4章

内存管理

本章内容

- ☐ 页
- ☐ 内存管理区
- ☐ 页面
- ☐ Slab 分配器
- ☐ Slab 分配器的生命周期
- ☐ 内存请求路径
- ☐ Linux 进程的内存结构
- ☐ 进程映像的分布及线性地址空间
- ☐ 页表
- ☐ 缺页

内存管理是运行在计算机上的应用程序通过软硬件协作来访问内存的一种方法。内存管理子系统的职责为：进程请求内存时分配可用内存，进程释放内存后回收内存，以及跟踪系统中内存的使用状况。

操作系统的生命周期划分为两个阶段：正常执行阶段和自举阶段。自举阶段使用临时内存；而正常运行阶段使用的内存有两种情况：一种是有一部分固定的内存分配给内核代码和数据，另一种是为动态内存请求分配内存。动态内存请求源于进程的创建和空间的扩张。本章要着重介绍操作系统正常运行时对内存的管理。

在深入研究内存管理的实现细节和各部分如何配合前，很有必要先理解一些有关内存管理的高级概念。本章首先介绍内存管理系统和虚拟内存的概念，然后介绍辅助内存管理的各种内核数据结构和算法。理解了内核如何管理内存之后，接着还要考虑进程内存是如何划分与管理的，并概括出它们是如何以自顶向下的方式组织到内核数据结构中的。最后当介绍完进程内存的获取、管理和释放后，我们会讨论缺页，以及上述这些概念在 PowerPC 和 x86 两种体系结构上的具体实现。

最简单的内存管理系统是运行进程对所有内存具有访问权的系统。以这样方式运行的进程必须包含对所需要的硬件进行操作的全部代码，必须能找到自己的内存地址，而且还必须能将自身的数据载入内存。这种方式不但给程序开发人员造成了很重的负担，而且还要保证进程与可用内存的大小合适。这些苛刻的要求对于日益复杂化的程序需求来说显然很不现实，所以要将内存管

理这个棘手的任务交给操作系统来对付，可用内存会在操作系统和用户进程之间划分。

当代操作系统既要求能够使多个程序共享系统资源，同时还要求内存限制对程序开发者透明。在此需求下，**虚拟内存**（Virtual memory）应运而生，虚拟内存支持程序访问比系统物理可用内存大得多的内存空间，而且也使得多个程序共享内存变得更高效。物理内存（或叫核心内存）是系统中由 RAM 芯片控制的可用内存。虚拟内存依靠透明地使用磁盘空间，得以使程序运行起来好像它们使用比系统物理内存更多的内存空间。磁盘空间（相比物理内存价格更低廉，容量也更大）可作为物理内存的扩充。我们之所以称其为虚拟内存就是因为磁盘存储体有效地充当内存，但它本身并不是内存。图 4-1 描述了多级数据存储体的层次关系。

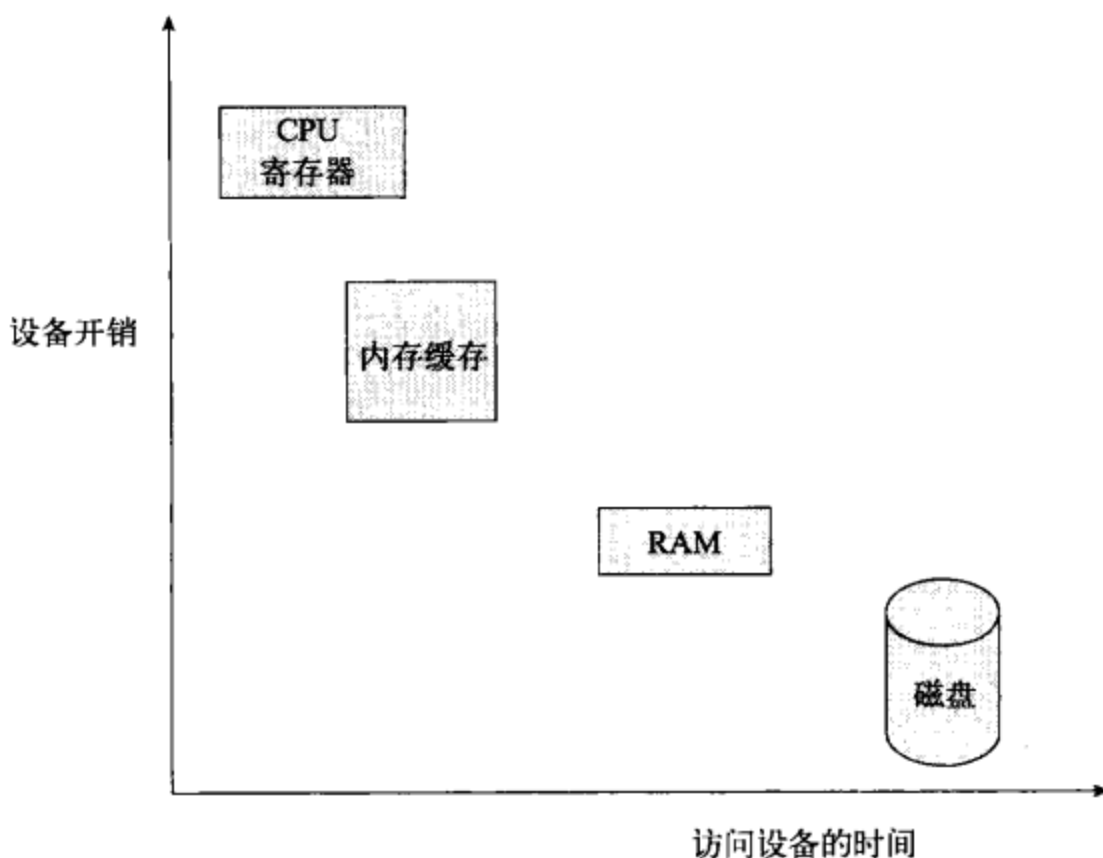


图 4-1 数据访问层次图

使用虚拟内存时，程序数据被分割成基本单元，这些单元可以在磁盘与内存间来回移动。这样，程序正使用的那部分就可以置于内存中，以便快速被访问，而未用的部分则被临时存放在磁盘，如此一来减轻待访问数据存在磁盘上导致读取时间过长的问题。这些数据单元（或者说虚拟内存块）被称作**页**（Page）。同理，物理内存也需要被划分成用于保存这些页的区，这些区被称作**页面**（Page Frame）。当进程请求访问一个地址时，该地址所在的页被载入内存，对页中任一数据的请求都会产生对该页的访问。如果页中的任一地址以前都没有被访问过，说明该页尚未被装入内存。对页中地址第一次访问便会产生一个**失败或缺页**（page fault），因为这时该页并没有在内存中，因此必须从磁盘请求。一次缺页便是一个**陷入**（trap）。当发生缺页时，内核必须选择一个页面，然后将其内容（页）写回到磁盘，从而用程序刚刚请求的页的内容来填充它。

当程序从内存中存取数据时，会使用地址来指出需要访问的内存位置。该地址被称作**虚拟地址**（virtual address），它们组成了进程**虚拟地址空间**（virtual address space）。每个进程都有自己独立的虚拟地址范围，这样做的好处是可防止非法读取或写覆盖其他进程的数据。虚拟内存允许

进程“使用”超过可用物理内存的内存空间，因此操作系统可以给每个进程提供独立的虚拟线性地址空间^①。

虚拟地址空间的大小取决于体系结构的字长，如果处理器的寄存器可容纳 32 位数值，那么运行该程序的处理器支持的进程虚拟地址空间由 2^{32} 个地址构成^②。虚拟内存不但扩大了可寻址的内存范围，而且也使得用户空间的开发者不必担心物理内存的本质所带来的限制，比如开发者不需要管理内存中的任何漏洞。以 32 位的系统为例，其虚拟地址空间的范围是 0~4 G，如果系统有 2 G 的物理内存，那么它的物理地址范围是 0~2 G。而程序可能有 4 GB 之巨，但是必须被装入可用内存中才能运行，因此整个程序将被存放在磁盘上，只有当需要时页才会被载入到内存。

将页在内存到磁盘之间调入调出的机制被称作**分页机制** (paging)，分页包括程序虚拟地址到物理内存地址的转换。

内存管理器在操作系统中负责维护虚拟地址和物理地址之间的关系，并且实现分页机制。对内存管理来说，页是基本的内存单元；MMU (Memory Management Unit，**内存管理单元**) 是完成实际的地址转换工作^③的硬件部件；内核提供了**页表** (对可用页进行索引的列表) 以及 MMU 在执行地址转换时要访问的相关地址。上述这些数据都会在页面载入内存时被更新。

我们了解内存管理的这些高层概念后，接下来开始分析内核如何实现内存管理器，先来看看页的实现。

4.1 页

作为内存管理器管理的基本内存单元，页的许多状态需要被记录下来。比如内核需要知道什么时候页可以被回收。为此，内核使用**页描述符**，内存中每个物理页都对应一个页描述符。

这一节阐述页描述符的各种字段，并对内存管理器如何使用这些字段给出说明。页结构定义在 include/linux/mm.h 文件中。

```
-----
include/linux/mm.h
170 struct page {
171     unsigned long flags;
172
173     atomic_t count;
174     struct list_head list;
175     struct address_space *mapping;
176     unsigned long index;
177     struct list_head lru;
178
179     union {
180         struct pte_chain *chain;
```

① 关于进程内存的使用，进程寻址做出几条假设：第一，进程不必同时使用其请求的所有内存；第二，从一个普通的可执行文件实例化而产生的两个或多个进程只需要将可执行对象载入内存一次。

② 虽然从技术上讲可用内存的大小界限是物理内存加上 swap 空间，但可寻址范围则取决于体系结构的字长。这意味着即便系统中有超过 4 GB 的内存，进程分配的内存也不超过 3 GB (要知道顶端 1 GB 是分配给内核的空间)。

③ 有些微处理器，比如 Motorola 68000 (68 K)，缺少 MMU 单元。uClinux 是一种特殊的 Linux 发布版，它被专门移植到非 MMU 系统中运行。它没有 MMU，因此虚拟地址等同于物理地址。

```

181
182     pte_addr_t direct;
183 } pte;
184 unsigned long private;
185
...
196 #if defined(WANT_PAGE_VIRTUAL)
197     void *virtual;
198
199 #endif
200 };
-----

```

标志

原子标志描述页面的状态。每个标志都由 32 位数值中的一位表示。系统提供一组辅助函数，用于操作和测试特定的标志，同时还可以通过一组辅助函数访问对应于特定标志的位的值。这些标志和辅助函数都定义于 `include/linux/page-flags.h` 文件中，表 4-1 给出并解释了可在页结构的 `flags` 字段中设置的一些标志。

表4-1 page->flags中的标志值

标 志 名	描 述
PG_locked	表示页被锁定，因此不可访问。该位用于保护磁盘I/O操作，在I/O操作前设置，I/O操作完成后清除
PG_error	表示这一页有I/O错误发生
PG_referenced	表示这一页在磁盘I/O操作期间被访问过。该标记可以决定页应位于哪个活动页链表上或非活动页链表上
PG_uptodate	表示页内容是有效的，当在页上的读操作完成后，该标志被设置。该标志与PG_error标志互斥
PG_dirty	表示被修改的页
PG_lru	表示该页是某个用于页交换的LRU（最近最少使用）链表。请看本节中对页结构lru字段的描述，从而了解更多关于LRU链表的信息
PG_active	表示页处于活动页链表中
PG_slab	表示页属于由slab分配器创建的Slab。其详细描述请见4.4节
PG_highmem	表示页位于高端内存区（ZONE_HIGHMEM）中。它不可被永久映射到内核虚拟地址空间中，高端内存区中的页在内核启动时由mem_init()函数打上该标志（请看第8章了解具体细节）
PG_checked	由ext2文件系统使用，但在2.5版内核中摒弃了该标志
PG_arch_1	与体系结构相关的页状态位
PG_reserved	表示该页不可被换出、不存在，或者已被启动内存分配器分配
PG_private	表示该页是有效的。当page->private包含有效值时，设置该标志
PG_writeback	表示页正在被写回
PG_mappedtodisk	表示页中含有当前在系统磁盘上分配的块区
PG_reclaim	表示页应该被回收
PG_compound	表示该页是高级混合页的一部分

1. count

count 字段相当于计数器，统计一个页使用或引用的次数。数值 0 表示页面是可重用的；正数表示访问该页数据^①的进程数。

2. list

list 字段是 list_head 结构，结构中的 next 和 prev 指针指向双向链表中的对应元素。这个页是否是双向链表的成员，其部分因素是由与页相关的映射和页状态确定的。

3. mapping

当页中存放的数据对应着文件的内存映射时，其中每个页就关联一个 address_space 结构。mapping 字段是指向 address_space 结构的指针，而页是属于该结构的成员。address_space 其实是属于内存对象（比如索引节点）的页面集合。有关如何使用 address_space 的详细信息，请参见第 7 章。

4. lru

lru 字段存放的 next 和 prev 指针指向最近最少使用链表中的相应元素。这些链表用在页面回收处理中，包括两个相应的链表：活动链表（active_list）和非活动链表（inactive_list），活动链表包含在用的页面，非活动链表包含可重用的页面。

5. virtual

virtual 是一个指向页所对应虚拟地址的指针。在含有高端内存^②的系统中，内存映射可以动态完成，所以很有必要在需要时重新计算虚拟地址，在这种情况下，该值要被设置成 NULL。

混合页

混合页是一种高阶页。如果能让内核支持混合页，就需要在编译时启用“Huge TLB Page Support”项。一个混合页由多个页面组成，其中首页被称为“头”页，其余被称为“尾”页。所有的混合页都需要在其相应的 page->flags 中设置 PG_compound 位，而且 page->lru.next 指向头页。

4.2 内存管理区

并非所有页都会被平等对待。在有些计算机体系中，对内存中某一物理地址范围的用途是有限制的。比如在 x86 体系中，部分 ISA 总线只能寻址最开始 16 MB 的内存范围。虽在 PPC 没有此类限制，但内存管理区（memory zone）的概念还是被保留下来，用以简化与体系结构无关的代码。因此在 PPC 的与体系结构相关的代码中，这些区被设置为重叠的。当一个系统的物理内存大于它能寻址的线性地址空间时，还会存在另一些此类限制。

内存管理区是由页面（或叫物理页）组成，这意味着，页面的分配来自特定的内存管理区。在 Linux 系统中存在三个内存管理区：ZONE_DMA（用于 DMA 的页面），ZONE_NORMAL（具有虚拟映射的非 DMA 页），ZONE_HIGHMEM（这些页的地址不在虚拟地址空间中）。

① 当页中的数据不再使用或不再需要时，该页便是自由的。

② 高端内存指超过了虚拟地址范围的物理内存部分。请参见 4.2 节了解更多细节。

4.2.1 内存管理区描述符

与内核管理的所有对象一样，每个内存管理区都对应一个叫做区域（zone）的结构，其中存放有关内存管理区的所有信息。区域结构定义于/linux/mmzone.h 文件中。接下来进一步学习一下该结构中最常用的一些字段。

```
-----
include/linux/mmzone.h
66 struct zone {
...
70  spinlock_t    lock;
71  unsigned long  free_pages;
72  unsigned long  pages_min, pages_low, pages_high;
73
74  ZONE_PADDING(_pad1_)
75
76  spinlock_t    lru_lock;
77  struct list_head active_list;
78  struct list_head inactive_list;
79  atomic_t    refill_counter;
80  unsigned long  nr_active;
81  unsigned long  nr_inactive;
82  int    all_unreclaimable; /* All pages pinned */
83  unsigned long  pages_scanned; /* since last reclaim */
84
85  ZONE_PADDING(_pad2_)
...
103  int temp_priority;
104  int prev_priority;
...
109  struct free_area free_area[MAX_ORDER];
...
135  wait_queue_head_t * wait_table;
136  unsigned long  wait_table_size;
137  unsigned long  wait_table_bits;
138
139  ZONE_PADDING(_pad3_)
140
...
157 } ____cacheline_maxaligned_in_smp;
-----
```

1. lock

当读写内存管理区描述符时，必须对其锁定以避免读/写错误。lock 字段存放的自旋锁便是保护描述符的。

该锁保护的是内存管理区描述符本身，而不是描述符所描述的内存范围。

2. free_pages

free_pages 字段存放内存管理区中剩余的空闲页个数。每当从某个内存管理区中分配一个页面，或者向内存管理区返回一个页面时，就对这个无符号长整数分别执行加 1 或者减 1 操作。

调用 `nr_free_pages()` 函数所返回的系统空闲总页数，就是系统中三个内存管理区中的该值相加的总和。

3. `pages_min`、`pages_low`和`pages_high`

`pages_min`、`pages_low` 和 `pages_high` 字段存放内存管理区的水位值（阈值）。当可用页面数跌到上述值时，意味着出现了内存紧张的紧急情况，此刻内核会根据相应规则做出响应。

4. `lru_lock`

`lru_lock` 字段存放的是空闲页列表的自旋锁。

5. `active_list`和`inactive_list`

`active_list` 和 `inactive_list` 两个链表用在页面回收处理时，前一个是处于活动状态的页链表，后一个是可被回收的页链表。

6. `all_unreclaimable`

如果内存管理区中所有页都被固定住，那么 `all_unreclaimable` 字段被置为 1。这些页只能被 `kswapd`（页换出守护进程）回收。

7. `pages_scanned`、`temp_priority`和`prev_priority`

`pages_scanned`、`temp_priority` 和 `prev_priority` 字段用于页面回收处理。它们的用法已经超出本书范围。

8. `free_area`

使用 `free_area` 位图的伙伴系统。

9. `wait_table`、`wait_table_size`和`wait_table_bits`

`wait_table`、`wait_table_size` 和 `wait_table_bits` 字段与内存管理区上页的进程等待队列相关。

缓存对齐和区填充

缓冲对齐是为了提高对描述符中字段的访问性能。缓冲对齐通过减少复制一组数据所需的指令数量来提高性能。假设有一个 32 位数值未按照字长对齐，那么处理器就需执行两次“载入字”的指令才把数据置入寄存器中，如果对齐的话，仅需要一次。`ZONE_PADDING` 字段描述内存管理区中缓冲的对齐方式。

```
-----
include/linux/mmzone.h
#ifdef CONFIG_SMP
struct zone_padding {
    int x;
} ____cacheline_maxaligned_in_smp;
#define ZONE_PADDING(name) struct zone_padding name;
#else
#define ZONE_PADDING(name)
#endif
-----
```

如果你想知道 Linux 中缓存对齐的工作原理，请参考 `include/linux/cache.h` 文件。

4.2.2 内存管理区操作辅助函数

当对一个对象进行一些常用的操作时，或者从对象中获取信息时，辅助函数通常可以简化编码。下面我们列出一些简化内存管理区操作的辅助函数。

1. `for_each_zone()`

宏 `for_each_zone()` 遍历系统中的所有内存管理区。

```
-----
include/linux/mmzone.h
268 #define for_each_zone(zone) \
269     for (zone = pgdat_list->node_zones; zone; zone = next_zone(zone))
-----
```

2. `is_highmem()`和`is_normal()`

`is_highmem()`和 `is_normal()` 两个函数检测内存管理区结构位于高端内存管理区中还是普通内存管理区中，其原型分别如下：

```
-----
include/linux/mmzone.h
315 static inline int is_highmem(struct zone *zone)
316 {
317     return (zone - zone->zone_pgdat->node_zones == ZONE_HIGHMEM);
318 }
319
320 static inline int is_normal(struct zone *zone)
321 {
322     return (zone - zone->zone_pgdat->node_zones == ZONE_NORMAL);
323 }
-----
```

4.3 页面

页面 (page frame) 是存放页的基本内存单元。只要进程请求内存，内核便会请求一个页面给它；同样道理，如果页面不再被使用，那么内核便将其释放，以便其他进程可以再使用。下面介绍的函数用来完成这些操作。

4.3.1 请求页面的函数

可以调用几个例程请求页面。我们依据其返回值的类型可以将它们分为两类。第一类返回指向 `page` 结构体的指针 (返回类型是 `void *`)，该结构体对应分配给请求者的页面。这类函数包括 `alloc_pages()` 和 `alloc_page()`。第二类函数返回 32 位虚拟地址 (返回类型是长整型)，该地址是所分配页面的首地址。这类函数包括 `__get_free_page()` 和 `__get_dma_pages()`。上述这些例程很多是对底层接口的包装。图 4-2 和图 4-3 描述这些例程的调用图。

接下来的宏和函数所请求或释放的页面数都是 2 的幂次方。也就是说，页面请求和释放都必须针对连续的页面，而且这些连续页面数目必须是 2 的幂次方。所以我们可以请求 1 页、2 页、4 页、8 页、16 页 (依次类推) 等数量页面^①。

① 页面的请求或释放总是连续的。

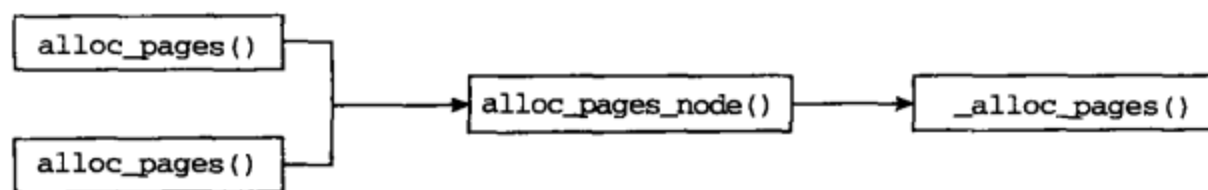


图 4-2 alloc_*()调用图

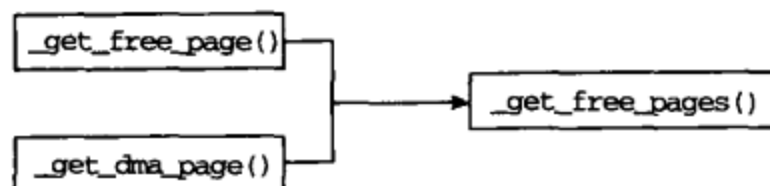


图 4-3 get_*_page()调用层次结构

1. alloc_pages()和alloc_page()

函数 alloc_page()用于请求单页，所以不需要说明内存的大小（不使用 order 参数）。该函数在调用 alloc_pages_node()时为 order 参数填充 0 值。而另一个函数 alloc_pages()则可以请求 4 个页面。

```

-----
include/linux/gfp.h
75  #define alloc_pages(gfp_mask, order) \
76    alloc_pages_node(numa_node_id(), gfp_mask, order)
77  #define alloc_page(gfp_mask) \
78    alloc_pages_node(numa_node_id(), gfp_mask, 0)
-----
  
```

正如我们在图 4-2 中所看到的，上述两个宏其实都是调用函数__alloc_pages_node()，并传递给它合适的参数。alloc_pages_node()其实是一个包装函数，用于对请求的页面数量进行合理性检查。

```

-----
include/linux/gfp.h
67  static inline struct page * alloc_pages_node(int nid, unsigned int gfp_mask, unsigned
    int order)
68  {
69    if (unlikely(order >= MAX_ORDER))
70      return NULL;
71
72    return __alloc_pages(gfp_mask, order, NODE_DATA(nid)->node_zonelist + (gfp_mask &
    GFP_ZONEMASK));
73  }
-----
  
```

如你所见，如果请求页面数量大于允许的最大数量（MAX_ORDER），那么页面分配的请求会被终止。在函数 alloc_page()中页面数量（order）总被设置为 0，所以请求总能得到满足。MAX_ORDER 定义于文件 linux/mmzone.h 中，其值是 11，因此最多可以请求 2 048 个页面。

函数__alloc_pages()是真正的处理页面请求的函数。该函数定义在文件 mm/page_alloc.c 中，而且和前面讨论的内存管理区知识相关。

2. `__get_free_page()`和`__get_dma_pages()`

宏`__get_free_page()`是请求单页操作的简化版本。它和`alloc_page()`一样，在调用`__get_free_pages()`时传入 0 作为请求的页面数量（函数`__get_free_pages()`用来执行多页面请求），图 4-3 描述了这些函数的调用层次关系。

```
-----
include/linux/gfp.h
83 #define __get_free_page(gfp_mask) \
84     __get_free_pages((gfp_mask), 0)
-----
```

宏`__get_dma_pages`指定从 `ZONE_DMA` 请求的页面，这是通过把 `GFP_DMA` 标志加到页标志掩码实现的。`ZONE_DMA` 指向的内存管理区是专为 DMA 访问所预留的。

```
-----
include/linux/gfp.h
86 #define __get_dma_pages(gfp_mask, order) \
87     __get_free_pages((gfp_mask) | GFP_DMA, (order))
-----
```

4.3.2 释放页面的函数

为了释放页面，内核提供了多个例程，包括两个宏和两个包装器函数。图 4-4 描述这些页释放例程之间的调用层次。依照它们的参数类型我们将这些函数分为两组。第一组函数为`__free_page()`和`__free_pages()`，其参数是指向待释放页的页描述符指针；第二组函数为`free_page()`和`free_pages()`，其参数为待释放页面的首地址。

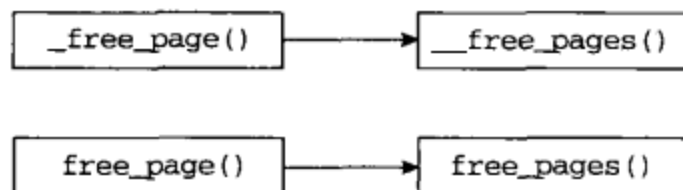


图 4-4 `*free_page*`()的调用层次

宏`__free_page()`和`free_page()`用于释放单页面。在调用页面释放函数时，将需释放的页面数量参数置为 0，函数`__free_pages()`和`free_pages()`的定义分别如下：

```
-----
include/linux/gfp.h
94 #define __free_page(page) __free_pages((page), 0)
95 #define free_page(addr) free_pages((addr), 0)
-----
```

函数`free_pages()`最终调用函数`__free_pages_bulk()`，后者是 Linux 伙伴系统的页面释放函数。我们将在下一节中详细讨论伙伴系统。

4.3.3 伙伴系统

每当页面被分配和回收时，系统都要遇到名为外部碎片（external fragmentation）的内存碎

片问题。这是由于可用页面散布于整个内存空间中，即使系统可用页面总数足够多，但也无法分配大块连续页面。这就是说，可用页面会被一个或多个不连续的不可用页面拆开。减少外部碎片的途径很多，Linux 使用伙伴系统 (buddy system) 这样的内存管理算法的实现来达到该目的。

伙伴系统把内存中的空闲块组成链表。每个链表都指向不同大小的内存块，虽然大小不同，但都是 2 的幂次方。至于系统中有多少个链表，则取决于具体实现。从最小的空闲块链表中分配页面，这样保证了较大的连续块可留给更大的内存请求。当分配的块被释放时，伙伴系统搜索与所释放块大小相等的可用空闲内存块，如果找到相邻的空闲块，那么就将它们合并成两倍于自身大小的一个新内存块。这些可合并的内存块（所释放的块和与其相邻的空闲块）就称为伙伴，所以也就有了伙伴系统一说。之所以这么做，是因为内核需要确保只要页面被释放，就能获得更大的可用内存块。

现在来看看 Linux 中实现伙伴系统的具体函数，其中页面分配函数是 `__alloc_pages()` (`mm/page_alloc.c`)，页面回收函数是 `__free_pages_bulk()`：

```
-----
mm/page_alloc.c
585 struct page * fastcall
586 __alloc_pages(unsigned int gfp_mask, unsigned int order,
587               struct zonelist *zonelist)
588 {
589     const int wait = gfp_mask & __GFP_WAIT;
590     unsigned long min;
591     struct zone **zones;
592     struct page *page;
593     struct reclaim_state reclaim_state;
594     struct task_struct *p = current;
595     int i;
596     int alloc_type;
597     int do_retry;
598
599     might_sleep_if(wait);
600
601     zones = zonelist->zones;
602     if (zones[0] == NULL) /* no zones in the zonelist */
603         return NULL;
604
605     alloc_type = zone_idx(zones[0]);
606     ...
607     for (i = 0; zones[i] != NULL; i++) {
608         struct zone *z = zones[i];
609
610         min = (1<<order) + z->pages_low;
611         ...
612         if (rt_task(p))
613             min -= z->pages_low >> 1;
614
615         if (z->free_pages >= min ||
616             (!wait && z->free_pages >= z->pages_high)) {
```



```

622     page = buffered_rmqueue(z, order, gfp_mask);
623     if (page) {
624         zone_statistics(zonelist, z);
625         goto got_pg;
626     }
627 }
628 }
629
630 /* we're somewhat low on memory, failed to find what we needed */
631 for (i = 0; zones[i] != NULL; i++)
632     wakeup_kswapd(zones[i]);
633
634 /* Go through the zonelist again, taking __GFP_HIGH into account */
635 for (i = 0; zones[i] != NULL; i++) {
636     struct zone *z = zones[i];
637
638     min = (1<<order) + z->protection[alloc_type];
639
640     if (gfp_mask & __GFP_HIGH)
641         min -= z->pages_low >> 2;
642     if (rt_task(p))
643         min -= z->pages_low >> 1;
644
645     if (z->free_pages >= min ||
646         (!wait && z->free_pages >= z->pages_high)) {
647         page = buffered_rmqueue(z, order, gfp_mask);
648         if (page) {
649             zone_statistics(zonelist, z);
650             goto got_pg;
651         }
652     }
653 }
...
720 nopage:
721     if (!(gfp_mask & __GFP_NOWARN) && printk_ratelimit()) {
722         printk(KERN_WARNING "%s: page allocation failure."
723             " order:%d, mode:0x%x\n",
724             p->comm, order, gfp_mask);
725         dump_stack();
726     }
727     return NULL;
728 got_pg:
729     kernel_map_pages(page, 1 << order, 1);
730     return page;
731 }

```

Linux 伙伴系统按内存管理区管理，这就意味可用页面链表也要按区分别维护，因此，可用页面的搜索可能发生在三个内存管理区中。

第 586 行：整数值 `gfp_mask` 让函数 `__alloc_pages()` 的调用者指定搜索页面的方式（操作修饰符）。该变量可能的取值定义在 `/linux/gfp.h` 中，表 4-2 列举了取值的详细情况。

表4-2 页面分配方式（操作修饰符gfp_mask）

标 志	描 述
__GFP_WAIT	允许内核阻塞进程等待页面。比如page_alloc.c中第537行使用到了它
__GFP_COLD	请求缓冲调清页面（cache cold page）
__GFP_HIGH	页面可从应急内存池中找到
__GFP_IO	页面可用于执行I/O传输
__GFP_FS	允许页面调用低级FS操作
__GFP_NOWARN	页面分配失败后，分配函数就发送一个失败警告，如果设置该修饰符，那么禁止警告。其用法见page_alloc.c中第665~666行
__GFP_REPEAT	请求失败后尝试再分配
__GFP_NORETRY	请求失败后不再重复
__GFP_DMA	页面处于ZONE_DMA[X]
__GFP_HIGHMEM	页面处于ZONE_HIGHMEM[X]

表 4-3 提供了指向内存管理区链表（zonelist）的指针，内存管理区链表的修饰符来自 gfp_mask。

表4-3 内存管理区链表

标 识	描 述
GFP_USER	表示不应该在内核内存中分配内存
GFP_KERNEL	表示将从内核内存中分配内存
GFP_ATOMIC	中断处理程序中调用kmalloc需设置该标志，它标识内存分配时调用者不可睡眠
GFP_DMA	表示将从ZONE_DMA区分配内存

第 599 行：函数 `might_sleep_if()` 获取 `wait` 变量的值（`wait` 是 `gfp_mask` 和 `__GFP_WAIT` 的逻辑与所得）。如果未设置 `__GFP_WAIT`，则 `wait` 值为 0；如果设置 `__GFP_WAIT`，则该值为 1。如果编译内核时检查到自旋锁内睡眠（Sleep-inside-spinlock）被启用（在内核编译选项的 Kernel Hacking 菜单下），那么该函数将允许内核将当前进程阻塞指定的超时时间。

第 608~628 行：在该代码段中，我们遍历内存管理区描述符链表，搜索一个可以为请求提供足够空闲页面的内存管理区。如果区中的空闲页面数可满足要求或如果请求进程允许等待，而且区中空闲页面数大于或等于区的上限值，那么就要调用函数 `buffered_rmqueue()`。

函数 `buffered_rmqueue()` 有三个参数：可用页面的内存管理区描述符，请求的页面数，请求页面的动作修饰符。

第 631~632 行：如果进入该代码段，说明我们将无法分配页面，因为可用页面数已经处于很低的状态，这时要做的是尝试回收页面来满足需要。函数 `wakeup_kswapd()` 实现这一功能，它会给内存管理区补充适量的页面，并适当更新区描述符。

第 635~653 行：当我们在前面代码段中尝试补充页面后，要重新进入内存管理区链表，再次搜寻满足请求的足够多的空闲页面。

第 720~727 行：在确定无页面可用的情况下才进入该代码段。如果未设置修饰符 `GFP_NOWARN`，那么就打印一个页面分配失败的警告，警告信息包括调用当前进程的命令名、所请求的页面大小、应用到这个请求的 `gfp_mask` 等信息。然后函数返回 `NULL`。

第 728~730 行：找到所要的页面后进入该代码段。函数返回一个页描述符的地址，如果请求超过一页，则返回所分配的首页描述符的地址。

当内存块被回收时，伙伴系统检查内存管理区中是否存在与该内存块同样大小的伙伴。如果存在，那么伙伴系统将它们合并成较大的内存块。函数 `__free_pages_bulk()` 便是执行该操作的。我们看看它如何工作：

```
-----
mm/page_alloc.c
178 static inline void __free_pages_bulk (struct page *page, struct page *base,
179     struct zone *zone, struct free_area *area, unsigned long mask,
180     unsigned int order)
181 {
182     unsigned long page_idx, index;
183
184     if (order)
185         destroy_compound_page(page, order);
186     page_idx = page - base;
187     if (page_idx & ~mask)
188         BUG();
189     index = page_idx >> (1 + order);
190
191     zone->free_pages -= mask;
192     while (mask + (1 << (MAX_ORDER-1))) {
193         struct page *buddy1, *buddy2;
194
195         BUG_ON(area >= zone->free_area + MAX_ORDER);
196         if (!__test_and_change_bit(index, area->map))
197             ...
206         buddy1 = base + (page_idx ^ ~mask);
207         buddy2 = base + page_idx;
208         BUG_ON(bad_range(zone, buddy1));
209         BUG_ON(bad_range(zone, buddy2));
210         list_del(&buddy1->lru);
211         mask <=> 1;
212         area++;
213         index >>= 1;
214         page_idx &= mask;
215     }
216     list_add(&(base + page_idx)->lru, &area->free_list);
217 }
```

第 184~215 行：函数 `__free_pages_bulk()` 遍历所有空闲块链（内存块最大不可超过 2 的 `MAX_ORDER` 次幂）。查遍所有空闲块，如果已经到达块的最大尺寸或该函数找到最小的伙伴后，就会调用函数 `__test_and_change_bit()` 来测试看我们所释放块的伙伴是否已被分配。如果伙伴页已被分配，那么跳出循环；如果还没被分配，那么则查看是否可以找到更大的伙伴块与页面的释放块合并。

第 216 行：将空闲块插入到合适的空闲页面链中。

4.4 Slab 分配器

我们已经知道页是内存管理的基本单元。但是进程往往会以字节为单位请求内存，而非以页

为单位请求内存。为了支持这种小块内存请求（比如调用 `kmalloc()`），内核特别实现了 slab 分配器（slab allocator），slab 分配器是一种对所请求的页进行管理的内存管理层。

Slab 分配器为了减少内存分配、初始化、销毁和释放的代价，通常会维护经常使用的内存区的一个现成的缓存区。这个缓存区维护着分配的、初始化的以及准备部署的内存区。当请求进程不再需要内存区时，就会把释放的内存送回缓存区中。

实际上，slab 分配器由许多缓存组成，不同的缓存存储大小不同的内存区。缓存可以是专用的（specialized），也可能是通用的（general purpose），专用缓存存储保存特定对象的内存区，比如各种描述符，像进程描述符（`task_structs`）就存放在 slab 分配器维护的缓存中，该缓存存储的内存区的大小为 `task_struct` 结构体的大小 `sizeof(task_struct)`。同样，`inode` 和 `dentry` 数据结构也存放在缓存中。一般来讲，通用缓存是预定义大小的内存区组成，其大小可以是：32、64、128、256、512、1 024、2 048、4 096、8 192、16 384、32 768、65 536 和 131 072 字节^①。

如果执行命令 `cat /proc/slabinfo`，系统中现有的 slab 分配器缓存都显示出来。在输出结果的第一列，可看到很多内核数据结构的名称和一组 `size-*` 格式的表项。前者对应存放特定对象的专用缓存，后者对应存放大小固定的通用对象的通用缓存。

你可能已经注意到任何大小的通用缓存都有两项，其中一个是以（DMA）结束。之所以有两个类似项，是由于既可从 DMA 内存管理区请求内存区^②，也可以从普通内存管理区请求，所以 slab 分配器同时维护了上述两种内存管理区对应的缓存，从而方便满足各种内存请求。图 4-5 显示了 `/proc/slabinfo` 的输出信息，从中就可看到两种类型内存的缓存。

size-131072 (DMA)	0	0	131072	1	32	tunables	8	4	0	slabdata	0	0
size-131072	0	0	131072	1	32	tunables	8	4	0	slabdata	0	0
size-65536 (DMA)	0	0	65536	1	16	tunables	8	4	0	slabdata	0	0
size-65536	1	1	65536	1	16	tunables	8	4	0	slabdata	1	1
size-32768 (DMA)	0	0	32768	1	8	tunables	8	4	0	slabdata	0	0
size-32768	0	0	32768	1	8	tunables	8	4	0	slabdata	0	0
size-16384 (DMA)	0	0	16384	1	4	tunables	8	4	0	slabdata	0	0
size-16384	0	0	16384	1	4	tunables	8	4	0	slabdata	0	0
size-8192 (DMA)	0	0	8192	1	2	tunables	8	4	0	slabdata	0	0
size-8192	64	68	8192	1	2	tunables	8	4	0	slabdata	64	68
size-4096 (DMA)	0	0	4096	1	1	tunables	24	12	0	slabdata	0	0
size-4096	65	65	4096	1	1	tunables	24	12	0	slabdata	65	65
size-2048 (DMA)	0	0	2048	2	1	tunables	24	12	0	slabdata	0	0
size-2048	102	102	2048	2	1	tunables	24	12	0	slabdata	51	51
size-1024 (DMA)	0	0	1024	4	1	tunables	54	27	0	slabdata	0	0
size-1024	73	100	1024	4	1	tunables	54	27	0	slabdata	25	25
size-512 (DMA)	0	0	512	8	1	tunables	54	27	0	slabdata	0	0
size-512	288	288	512	8	1	tunables	54	27	0	slabdata	36	36
size-256 (DMA)	0	0	256	15	1	tunables	120	60	0	slabdata	0	0
size-256	149	165	256	15	1	tunables	120	60	0	slabdata	11	11
size-128 (DMA)	0	0	128	30	1	tunables	120	60	0	slabdata	0	0
size-128	4906	10290	128	30	1	tunables	120	60	0	slabdata	343	343
size-64 (DMA)	0	0	64	58	1	tunables	120	60	0	slabdata	0	0
size-64	1565	2323	64	58	1	tunables	120	60	0	slabdata	40	40

图 4-5 `cat /proc/slabinfo`

① 为了提高性能，所有通用缓存都是 L1 对齐的。

② 注意区别内存管理区（zone）和内存区（memory area）。——译者注

可以把缓存进一步划分成所谓 slab 的容器，每个 slab 都由一个或多个连续的页面组成，小内存区便从 slab 容器这里分配，这就是为什么我们说 slab 容纳的是对象。对象本身便是属于 slab 的页面中预定义大小的地址区间，图 4-6 显示了 slab 分配器的组织关系。

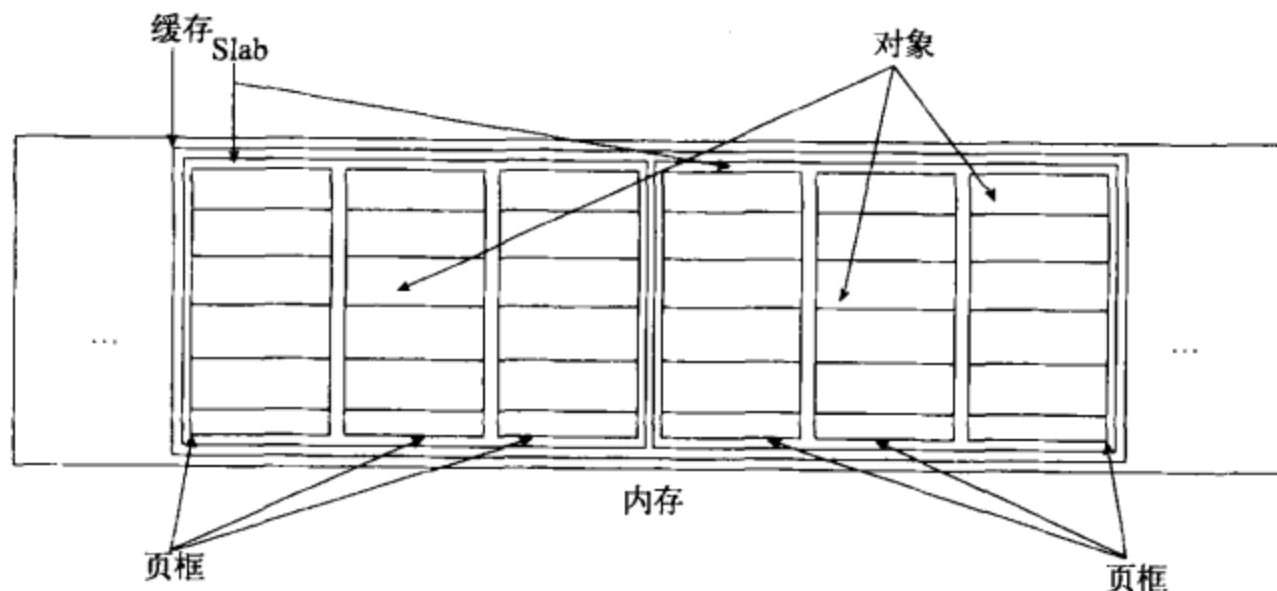


图 4-6 Slab 分配器组织关系图

Slab 分配器使用三个主要结构维护对象信息，它们分别是：称为 `kmem_cache` 的缓存描述符，称为 `cache_sizes` 的通用缓存描述符和称为 `slab` 的 slab 描述符。图 4-7 总结了所有这些描述符之间的逻辑关系。

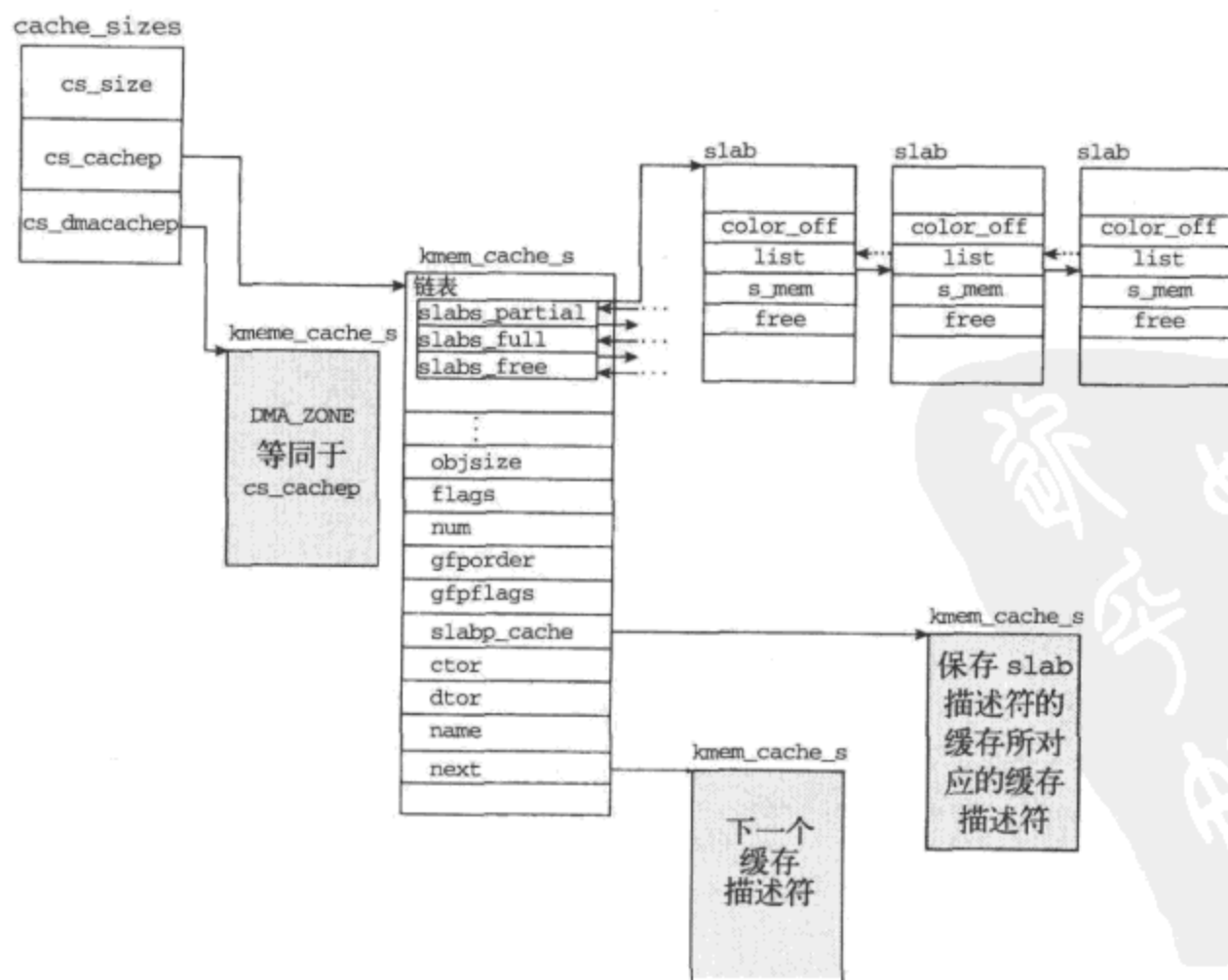


图 4-7 Slab 分配器结构

4.4.1 缓存描述符

每一个缓存都有一个类型为 `kmem_cache_s` 的缓存描述符，描述符中包含缓存的许多信息。这些信息中大多数值都是由函数 `kmem_cache_create()` (`mm/slab.c`) 在缓存创建时设置或计算获得的。我们会在后面的章节中讨论它们，这里我们首先来看看缓存描述符中的一些字段以及它们存储的信息。

```
-----
mm/slab.c
246 struct kmem_cache_s {
...
252 struct kmem_list3 lists;
...
254 unsigned int objsize;
255 unsigned int flags; /* constant flags */
256 unsigned int num; /* # of objs per slab */
...
263 unsigned int gfporder;
264
265 /* force GFP flags, e.g. GFP_DMA */
266 unsigned int gfpflags;
267
268 size_t color; /* cache coloring range */
269 unsigned int color_off; /* color offset */
270 unsigned int color_next; /* cache coloring */
271 kmem_cache_t *slabp_cache;
272 unsigned int dflags; /* dynamic flags */
273
273 /* constructor func */
274 void (*ctor)(void *, kmem_cache_t *, unsigned long);
275
276 /* de-constructor func */
277 void (*dtor)(void *, kmem_cache_t *, unsigned long);
278
279 /* 4) cache creation/removal */
280 const char *name;
281 struct list_head next;
282
...
301 };
-----
```

1. list

`list` 字段中包含三个链表头，其中每个链表头对应 `slab` 所处的三种状态之一：部分、完全和空闲。一个缓存可以有一个或多个 `slab` 处于上述任何一个状态中。缓存正是通过这个数据结构引用 `slab`。`list` 本身是由 `slab` 描述符的 `list` 字段维护着的双向链表。有关“`slab` 描述符”的内容将在本章后续部分陆续讲到。

```
-----
mm/slab.c
217 struct kmem_list3 {
218 struct list_head slabs_partial;
219 struct list_head slabs_full;
220 struct list_head slabs_free;
...

```

```

223 unsigned long next_reap;
224 struct array_cache *shared;
225 };

```

● lists.slabs_partial

lists.slabs_partial 是指 slab 中只有部分对象被分配的链表头，也就是说，如果 slab 处在“部分”状态下，表示它有一部分对象被分配，还有一部分空闲待用。

● lists.slabs_full

lists.slabs_full 是所有对象都已被分配的 slab 链表头，也就是说，slab 中没有可用对象。

● lists.slabs_free

lists.slabs_free 是指所有对象都空闲的 slab 链表头，也就是说，该 slab 中所有的对象都可用。

维护这些链表的目的是为了缩减寻找空闲对象的时间。当请求缓存中的对象时，内核将搜索部分 slab 链，如果部分 slab 链是空的，那么将搜索空闲 slab，如果空闲 slab 链表是空，那么将创建一个新的 slab。

● lists.next_reap

slab 存放分配给它们的页面，如果这些页并未使用，那最好将它们退回给主内存池。最后，缓存也会被收回。该字段存放的是下一个缓冲回收的时间，在调用函数 kmem_cache_create() (mm/slab.c) 创建缓存时设置该字段，并且每次调用函数 cache_reap() 时更新该字段。

2. objsize

objsize 字段存放缓存中对象的大小（以字节为单位），该值取决于缓存创建时请求的大小以及对齐的需要。

3. flag

flag 字段存放标志掩码，掩码描述缓存的固有特性。include/linux/slab.h 文件中定义了所有可能的标志。表 4-4 详细描述这些标志的含义。

表4-4 slab标志

flag 名称	描 述
SLAB_POISON	要求在创建slab时将a5a5a5a5作为测试数据写入slab中。该标志用于验证已初始化的内存
SLAB_NO_REAP	当出现内存不足时，内存管理器将开始回收没有使用的内存。设置该标志将保证该缓存不会在此情况下被自动回收
SLAB_HWCACHE_ALIGN	要求对象与处理器的硬件缓存线（cacheline）对齐，以便减少内存访问周期，从而提高系统访问性能
SLAB_CACHE_DMA	表示将使用DMA内存。当请求新页面时，GFP_DMA标志会传给伙伴系统
SLAB_PANIC	表示不管因为什么原因，如果函数kmem_cache_create()失败，则调用panic

4. num

num 字段存放缓存中每个 slab 所包含的对象数目。它是在缓存创建（也使用 kmem_cache_create()）时，根据 gfporder 的值（详见下个字段）、需创建对象的大小以及对齐要求确定的。

5. gfporder

`gfporder` 是缓存中每个 slab 所占连续页面数的幂（基数为 2）。值默认为 0，缓存创建时调用函数 `kmem_cache_create()` 时设置该值。

6. gfpflags

`gfpflags` 标志指定缓存中为 slab 分配的页面类型。这些类型由内存区请求的标志决定，比如，如果是 DMA 将使用的内存，那么 `gfpflags` 字段将被设置为 `GFP_DMA`，而且该字段将在请求页面时作为参数被传入。

7. slabp_cache

slab 描述符可以存放在缓存自身内，也可存放在外部其他缓存中。如果这个缓存中 slab 的描述符存放在自身以外的缓存中，那么 `slabp_cache` 字段保存指向缓存所对应的缓存描述符的指针，这些缓存存储 Slab 描述符对象。请看 4.4.3 节，了解更多 slab 描述符存放的信息。

8. ctor

如果存在构造函数，则 `ctor` 字段指向与缓存相关的构造函数的指针^①。

9. dtor

与 `ctor` 字段类似，如果存在析构函数，则 `dtor` 字段指向与缓存相关的析构函数的指针。

构造函数和析构函数都在缓存创建时定义，而且作为参数传递给函数 `kmem_cache_create()`。

10. name

`name` 字段存放可读性好的字符串的名称，可以在打开 `/proc/slabinfo` 时显示。比如对于存放文件指针的缓存，则该字段的值为 `filp`。调用 `cat /proc/slabinfo` 能更好地理解这一点。`name` 字段存放的名称要唯一。在创建时为 Slab 指定的名称，要与链表中所有其他 slab 的名称进行比较，看是否有重名。如果发现有相同名称的 slab 已存在，那么 slab 创建失败。

11. next

`next` 字段是指向缓存描述符单链表中下一个缓存描述符的指针。

4.4.2 通用缓存描述符

前面我们提到，通用缓存总是成对出现，其中存放预定大小的对象。一个缓存从 DMA 内存区分配对象，另一个缓存从普通内存区中分配。如果你还记得前面提到的内存管理区，就应该知道 DMA 缓存是在 `ZONE_DMA` 区中，而标准缓存则来自 `ZONE_NORMAL` 区。`struct cache_sizes` 存放通用缓存大小的所有信息。

```
-----
include/linux/slab.h
69 struct cache_sizes {
70     size_t    cs_size;
71     kmem_cache_t *cs_cachep;
72     kmem_cache_t *cs_dmacachep;
73 };
-----
```

^① 对于熟悉面向对象程序开发的读者来说，构造函数和析构函数都不陌生。缓存描述符的 `ctor` 字段让构造函数在每次创建新缓存描述符时被调用。类似地，`dtor` 字段存放指向每次销毁缓存描述符时调用的析构函数的指针。

1. cs_size

cs_size 字段存放该缓存中所包含的内存对象大小。

2. cs_cachep

cs_cachep 字段存放指向普通内存缓存描述符的指针，这种缓存存放的对象分配自 ZONE_NORMAL 内存区。

3. cs_dmacachep

cs_cachep 字段存放指向 DMA 内存缓存描述符的指针，DMA 内存缓存存放的对象分配自 ZONE_DMA。

现在有个问题，“缓存描述符本身存放在哪里？”slab 分配器有一个缓存专门预留来解决该问题。cache_cache 缓存便是用来存放缓存描述符对象的。slab 缓存是在系统自举时被静态初始化，以确保缓存描述符存储空间可用。

4.4.3 Slab 描述符

在缓存中的每个 slab 都包含针对该 slab 特定信息的描述符，我们已经提到，缓存描述符存放在叫做 cache_cache 的专用缓存中。而 slab 描述符可依次存放在两个地方：在 slab 本身中（特指在第一个页面中），或者在外部的第一个“通用”缓存中，当然这个缓存要足够大以容纳 slab 描述符。具体存放位置要根据在缓存创建时满足对象对齐要求后剩余空闲空间综合考虑。剩余空间要在缓存创建时确定。

下面来看看 slab 描述符的一些字段。

```
-----
mm/slab.c
173 struct slab {
174     struct list_head list;
175     unsigned long coloroff;
176     void *s_mem; /* including color offset */
177     unsigned int inuse; /* num of objs active in slab */
178     kmem_bufctl_t free;
179 };
-----
```

1. list

如果你回想一下关于缓存描述符的讨论，就可知道 slab 可以处于三种状态之一：free、partial 或 full。缓存描述符中存放处于全部三种状态的 slab 链表，每个状态对应一个链表，所有状态的 slab 都通过 list 字段保存在双向链表中。

2. s_mem

s_mem 字段存放的指针指向 slab 中的第一个对象。

3. inuse

inuse 值记录 slab 中含有对象的数量。对于满或部分满的 slab，该值为正值；对于空闲 slab，该值为 0。

4. free

free 字段存放 slab 对象数组的索引值，该字段所包含数组项的索引值，这个数组项表示 slab

中第一个可用对象。kmem_bufctl_t 数据类型链接处于同一 slab 中的所有对象，该数据类型仅仅是一个无符号整数，定义于文件 include/asm/types.h 中。这些数据类型组成了一个数组，且无论 slab 描述符存放在 slab 内部或外部，这个数组总是正好存放在 slab 描述符之后。要知详情，请察看内联函数 slab_bufctl()，该函数会返回对象数组。

```
-----
mm/slab.c
1614 static inline kmem_bufctl_t *slab_bufctl(struct slab *slabp)
1615 {
1616     return (kmem_bufctl_t *) (slabp+1);
1617 }
-----
```

函数 slab_bufctl() 接收一个指向 slab 描述符的指针，返回紧跟在 slab 描述符后的内存地址指针。

当缓存被初始化时，slab->free 字段被设置成 0（因为所有对象都是空闲的，因此应该返回第一个对象），而且 kmem_bufctl_t 数组中的每一项都设置为数组中下一个元素的索引值，这就是说第 0 个元素存放的值为 1，第一个元素存放的值为 2，以此类推。数组的最后一个元素存放值 BUFCTL_END，表示数组已经到头了。

图 4-8 描述了当 slab 描述符存放在自身内部时，slab 描述符、bufctl 数组以及 slab 对象的布局。表 4-5 显示了当 slab 处于三中可能状态时，slab 描述符字段可能的取值。

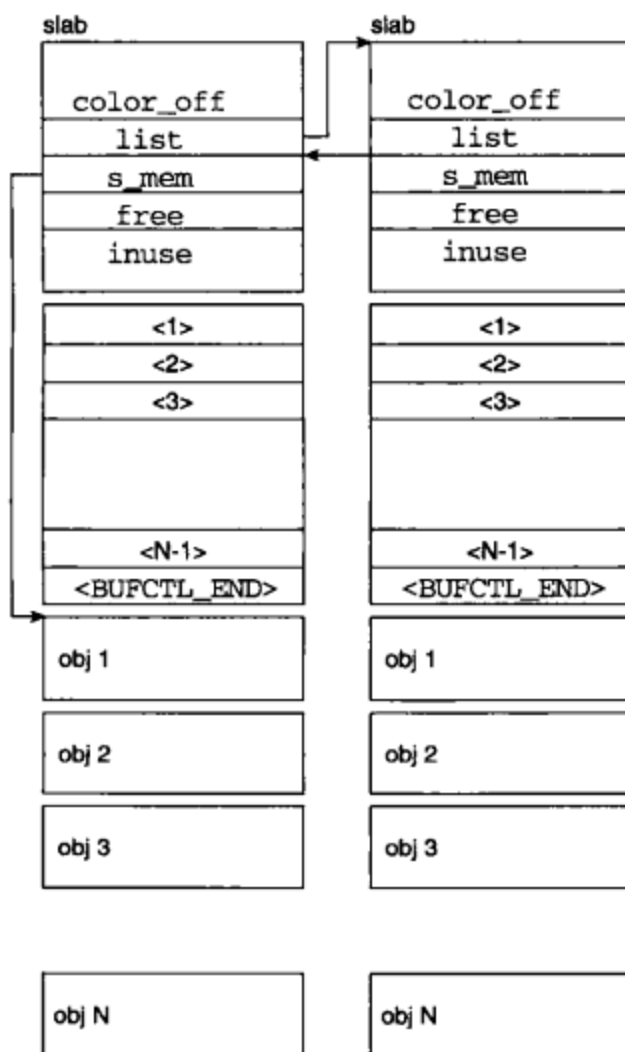


图 4-8 Slab 描述符和 bufctl

表4-5 Slab状态和描述符字段的值

	Free	Partial	Full
slab->inuse	0	X	N
slab->free	0	X	N

N = slab 中的对象数。

X = 某一可变的正数。

4.5 Slab 分配器的生命周期

现在，我们在内核运行的整个生命期范围内，观察缓存和 slab 分配器是如何交互的。内核需要确保某些结构正好能支持进程对内存区的请求，某些结构能支持动态可加载模块对专用缓存的动态创建。

因此，有一些全局结构对 slab 分配器具有关键作用。其中一些我们在前面章节未作介绍，这里我们来看看这些全局变量。

4.5.1 与 Slab 分配器有关的全局变量

有下面一组与 slab 分配器相关的全局变量。

- ❑ **cache_cache**: 它是一个特殊缓存的缓存描述符，其特殊性体现在该缓存存放的是其他所有缓存描述符。这个缓存的名称是 `kmem_cache`（在 `/proc/slabs/` 中的名字）。该缓存描述符是唯一一个静态分配的缓存描述符。
- ❑ **cache_chain**: 用作指向缓存描述符链表的指针的链表元素。
- ❑ **cache_chain_sem**: 对 `cache_chain` 进行控制访问的信号量^①。每次向链表中添加元素（新缓存描述符）时，分别调用 `down()` 和 `up()` 函数获取和释放信号量。
- ❑ **malloc_sizes[]**: 该数组存放 DMA 区的缓存描述符和与通用缓存对应的非 DMA 区的缓存描述符。

在 slab 分配器初始化之前，这些结构已经存在，让我们看看它们的创建过程：

```
-----
mm/slab.c
486 static kmem_cache_t cache_cache = {
487     .lists    = LIST3_INIT(cache_cache.lists),
488     .batchcount = 1,
489     .limit     = BOOT_CPUCACHE_ENTRIES,
490     .objsize   = sizeof(kmem_cache_t),
491     .flags     = SLAB_NO_REAP,
492     .spinlock  = SPIN_LOCK_UNLOCKED,
493     .color_off = L1_CACHE_BYTES,
494     .name      = "kmem_cache",
495 };
496
497 /* Guard access to the cache-chain. */
```

① 信号量的详细信息将在第9章讨论。

```

498 static struct semaphore cache_chain_sem;
499
500 struct list_head cache_chain;
-----

```

cache_cache 缓存描述符中若设置了 SLAB_NO_REAP 标志, 那么即使系统内存紧张, 该缓存也会在内核运行期间驻留内存。注意, cache_chain 信号量仅被定义, 并未初始化。初始化是在系统初始化时由函数 kmem_cache_init() 完成的。我们来看看该函数的实现细节:

```

-----
mm/slab.c
462 struct cache_sizes malloc_sizes[] = {
463 #define CACHE(x) { .cs_size = (x) },
464 #include <linux/kmalloc_sizes.h>
465 { 0, }
466 #undef CACHE
467 };
-----

```

这段代码初始化 malloc_sizes[] 数组, 并按照在 include/linux/kmalloc_sizes.h 文件中定义的值设置 cs_size 字段。正如我们以前提到的, 缓存大小的取值范围是 32 字节~131 072 字节, 具体值取决于内核配置^①。

依靠这些全局变量, 内核通过调用文件 init/main.c^② 中的函数 kmem_cache_init() 来继续初始化 slab 分配器。该函数负责初始化缓存链表、信号量、通用缓存、kmem_cache 缓存, 本质上也就是初始化管理 slab 的 slab 分配器用到的所有全局变量。之后, 专用缓存方可被创建。创建缓存的函数为 kmem_cache_create()。

4.5.2 创建缓存

创建缓存包含三个步骤:

- (1) 描述符的分配和初始化;
- (2) 计算 slab 着色和对象大小;
- (3) 在 cache_chain 链表中添加缓存。

通用缓存是由函数 kmem_cache_init() (mm/slab.c) 在系统初始化时创建的, 专用缓存则由函数 kmem_cache_create() 创建的。

我们分别来看看这两个函数。

1. kmem_cache_init()

该函数创建 cache_chain 和通用缓存, 它是在初始化过程中被调用的。注意该函数名前有一个 _init 前缀, 在第 2 章解释过, _init 修饰的函数载入内存后会在自举和初始化过程结束后被销毁。

① 内核选项中有一些额外的配置选项可支持超过 131 072 字节的通用缓存。更多信息可参阅文件 include/linux/kmalloc_sizes.h。

② 第 9 章将会把系统自启动上电开始的初始化过程——道来。我们将看到函数 kmem_cache_init() 如何在自举过程中发挥作用。

```

-----
mm/slab.c
659 void __init kmem_cache_init(void)
660 {
661     size_t left_over;
662     struct cache_sizes *sizes;
663     struct cache_names *names;
664     ...
669     if (num_physpages > (32 << 20) >> PAGE_SHIFT)
670         slab_break_gfp_order = BREAK_GFP_ORDER_HI;
671
672
-----

```

第 661~663 行：变量 `sizes` 和 `names` 是由 `kmalloc` 分配的数组（大小为几何分布的通用缓存）的头。此刻这些数组被置于 `_init` 数据区。注意 `kmalloc()` 此时还并不存在。函数 `kmalloc()` 使用我们刚刚建立的 `malloc_sizes` 数组。到此为止，我们所拥有的只有静态分配的 `cache_cache` 描述符。

第 669~670 行：这个代码段确定一个 slab 使用多少页。一个 slab 中的页面数完全取决于系统有多少可用内存。在 x86 和 PPC 体系机构中，变量 `PAGE_SHIFT` (`include/asm/page.h`) 值为 12，所以我们要确认 `num_physpages` 的值是否大于 8 K。如果超过的话说明我们的机器有超过 32 M 的内存，这时每个 slab 用 `BREAK_GFP_ORDER_HI` 个页；否则为每个 slab 分配 1 页。

```

-----
mm/slab.c
690     init_MUTEX(&cache_chain_sem);
691     INIT_LIST_HEAD(&cache_chain);
692     list_add(&cache_cache.next, &cache_chain);
693     cache_cache.array[smp_processor_id()] = &initarray_cache.cache;
694
695     cache_estimate(0, cache_cache.objsize, 0,
696         &left_over, &cache_cache.num);
697     if (!cache_cache.num)
698         BUG();
699
700
-----

```

第 690 行：该行初始化 `cache_chain` 所存放的信号量 `cache_chain_sem`。

第 691 行：初始化 `cache_chain` 链表，所有的缓存描述符都存放在该链表中。

第 692 行：向 `cache_chain` 链表中加入 `cache_cache` 描述符。

第 693 行：创建“每个 CPU”的缓存。其细节已经超出了本书范围。

第 695~698 行：该代码段执行完整性检查，也就是检查在 `cache_cache` 中是否至少分配了一个缓存描述符。而且该代码段还设置了 `cache_cache` 描述符的 `num` 字段，并计算还有多少剩余空间。这主要为了 slab 着色。slab 着色 (Slab Coloring) 是一种内核用于降低缓存对齐所带来的性能下降的方法。

```

-----
mm/slab.c
705  sizes = malloc_sizes;
706  names = cache_names;
707
708  while (sizes->cs_size) {
...
714      sizes->cs_cachep = kmem_cache_create(
715          names->name, sizes->cs_size,
716          0, SLAB_HWCACHE_ALIGN, NULL, NULL);
717      if (!sizes->cs_cachep)
718          BUG();
719
...
725
726      sizes->cs_dmacachep = kmem_cache_create(
727          names->name_dma, sizes->cs_size,
728          0, SLAB_CACHE_DMA|SLAB_HWCACHE_ALIGN, NULL, NULL);
729      if (!sizes->cs_dmacachep)
730          BUG();
731
732      sizes++;
733      names++;
734  }
-----

```

第 708 行：该行代码验证我们是否已经到达 `sizes` 数组的末尾，`sizes` 数组最后一个元素总被设置成 0，因此当我们到达数组最后一个元素时，判断条件就为真。

第 714~718 行：为常规内存分配请求创建下一个 `kmalloc` 缓存，并且验证该缓存是否为空。参见“`kmem_cache_create()`”部分。

第 726~730 行：这一代码段为 DMA 内存分配创建缓存。

第 732~733 行：找到 `sizes` 和 `names` 数组中的下一个元素。

函数 `kmem_cache_init()` 的剩余代码处理“用 `kmalloc` 分配的数据替换临时自举数据”的过程。我们省略介绍这一部分是因为它与实际的缓存描述符初始化没有直接关系。

2. `kmem_cache_create()`

当通用缓存所提供的内存区不能满足需要时，会调用该函数来创建专用缓存。创建专用缓存的步骤与创建通用缓存不大相同，依次包括：创建、分配、初始化缓存描述符、对齐对象、对齐 slab 描述符、添加缓存到缓存链表中。该函数没有 `__init` 前缀，因为它被调用后，可以使用永久内存。

```

-----
mm/slab.c
1027  kmem_cache_t *
1028  kmem_cache_create (const char *name, size_t size, size_t offset,
1029      unsigned long flags, void (*ctor)(void*, kmem_cache_t *, unsigned long),
1030      void (*dtor)(void*, kmem_cache_t *, unsigned long))
1031  {
1032      const char *func_nm = KERN_ERR "kmem_create: ";
1033      size_t left_over, align, slab_size;
1034      kmem_cache_t *cachep = NULL;
...
-----

```


我们来看看函数 `kmem_cache_create()` 的参数。

- name

用于标识缓存的名字。它存放在缓存描述符的 `name` 字段中，可以在文件 `/proc/slabinfo` 中看到这个名字。

- size

该参数指定缓存中包含的对象大小（以字节为单位），该值存放在缓存描述符的 `objsize` 字段中。

- offset

该值决定对象在一个页面上的位置。

- flags

`flags` 参数与 `slab` 相关。请参考表 4-4 对缓存描述符中 `flag` 字段和可能取值的描述。

- ctor和dtor

`ctor` 和 `dtor` 字段分别指向这个内存区中创建和销毁对象时调用的构造函数和析构函数。该函数执行大范围的调试与完整性检查，这里我们并未涉及。更详细的功能参见代码：

```
-----
mm/slab.c
1079  /* Get cache's description obj. */
1080  cachep = (kmem_cache_t *) kmem_cache_alloc(&cache_cache, SLAB_KERNEL);
1081  if (!cachep)
1082      goto opps;
1083  memset(cachep, 0, sizeof(kmem_cache_t));
1084
...
1144  do {
1145      unsigned int break_flag = 0;
1146      cal_wastage:
1147      cache_estimate(cachep->gfporder, size, flags,
1148                    &left_over, &cachep->num);
...
1174  } while (1);
1175
1176  if (!cachep->num) {
1177      printk("kmem_cache_create: couldn't create cache %s.\n", name);
1178      kmem_cache_free(&cache_cache, cachep);
1179      cachep = NULL;
1180      goto opps;
1181  }
-----
```

第 1079~1084 行：缓存描述符在此分配。此后的代码负责将 `slab` 中对象对齐。这里先不讨论对齐问题。

第 1144~1174 行：这段代码段决定缓存中对象的数量。大部分工作由函数 `cache_estimate()` 完成。计算出的对象数量最终会被存放在缓存描述符的 `num` 字段中。

```
mm/slab.c
...
1201  cachep->flags = flags;
1202  cachep->gfpflags = 0;
1203  if (flags & SLAB_CACHE_DMA)
1204    cachep->gfpflags |= GFP_DMA;
1205  spin_lock_init(&cachep->spinlock);
1206  cachep->objsize = size;
1207  /* NUMA */
1208  INIT_LIST_HEAD(&cachep->lists.slabs_full);
1209  INIT_LIST_HEAD(&cachep->lists.slabs_partial);
1210  INIT_LIST_HEAD(&cachep->lists.slabs_free);
1211
1212  if (flags & CFLGS_OFF_SLAB)
1213    cachep->slabp_cache = kmem_find_general_cachep(slab_size, 0);
1214  cachep->ctor = ctor;
1215  cachep->dtor = dtor;
1216  cachep->name = name;
1217
...
1242
1243  cachep->lists.next_reap = jiffies + REAPTIMEOUT_LIST3 +
1244    ((unsigned long)cachep)%REAPTIMEOUT_LIST3;
1245
1246  /* Need the semaphore to access the chain. */
1247  down(&cache_chain_sem);
1248  {
1249    struct list_head *p;
1250    mm_segment_t old_fs;
1251
1252    old_fs = get_fs();
1253    set_fs(KERNEL_DS);
1254    list_for_each(p, &cache_chain) {
1255      kmem_cache_t *pc = list_entry(p, kmem_cache_t, next);
1256      char tmp;
...
1265      if (!strcmp(pc->name, name)) {
1266        printk("kmem_cache_create: duplicate cache %s\n", name);
1267        up(&cache_chain_sem);
1268        BUG();
1269      }
1270    }
1271    set_fs(old_fs);
1272  }
1273
1274  /* cache setup completed, link it into the list */
1275  list_add(&cachep->next, &cache_chain);
1276  up(&cache_chain_sem);
1277  opps:
1278  return cachep;
1279 }
```

在此之前，slab 已经与硬件缓存对齐并进行了着色。slab 描述符中的 color 和 color_off 字段已被填充。

第 1200~1217 行：该代码段初始化缓存描述符的各个字段，与我们在 kmem_cache_init() 函数中看到的类似。

第 1243~1244 行：设置下一次回收缓存的时间。

第 1247~1276 行：初始化缓存描述符，并计算和存放与缓存相关的所有信息。现在我们将新的缓存描述符加入 cache_chain 链表中。

4.5.3 创建 slab 与 cache_grow()

当缓存刚被创建时，其中没有 slab。事实上，只有当一个对象发出请求的确需要 slab 时才真正分配 slab。这种情况出现在缓存描述符的 list.slabs_partial 和 lists.slabs_free 字段都为空时。此时我们并不关心对内存的请求如何转化成对特定缓存中对象的请求，而是假定这种转化已经发生，把关注点转移到 slab 分配器内的具体实现上。

函数 cache_grow() 创建 slab 并将其放在在缓存中。当我们创建 slab 时，不但分配和初始化其描述符，而且还需要分配物理内存，为此，我们需要与伙伴系统交互以请求页面。这个工作是由函数 kmem_getpages() (mm/slab.c) 完成的。

cache_grow()

函数 cache_grow() 用来在缓存中增加一个 slab，当缓存中没有空闲对象可用时就调用该函数。这种情况发生在 lists.slabs_partial 和 lists.slabs_free 都为空时。

```
-----
mm/slab.c
1546 static int cache_grow (kmem_cache_t * cachep, int flags)
1547 {
...
-----
```

传给该函数的参数有下面两个。

❑ **cachep**：它是需要扩充的缓存对应的缓存描述符。

❑ **flags**：这些标志将用于创建 slab。

```
-----
mm/slab.c
1572 check_irq_off();
1573 spin_lock(&cachep->spinlock);
...
1581
1582 spin_unlock(&cachep->spinlock);
1583
1584 if (local_flags & __GFP_WAIT)
1585     local_irq_enable();
-----
```

第 1572~1573 行：禁用中断且锁定描述符，为操作缓存描述符的字段做准备。

第 1582~1585 行：解锁缓存描述符且重新启用中断。

```
-----
mm/slab.c
...
1597  if (!(objp = kmem_getpages(cachep, flags)))
1598      goto failed;
1599
1600  /* Get slab management. */
1601  if (!(slabp = alloc_slabmgmt(cachep, objp, offset, local_flags)))
1602      goto opps1;
...
1605  i = 1 << cachep->gfporder;
1606  page = virt_to_page(objp);
1607  do {
1608      SET_PAGE_CACHE(page, cachep);
1609      SET_PAGE_SLAB(page, slabp);
1610      SetPageSlab(page);
1611      inc_page_state(nr_slab);
1612      page++;
1613  } while (--i);
1614
1615  cache_init_objs(cachep, slabp, ctor_flags);
-----
```

第 1597~1598 行：与伙伴系统交互以获取页面存放 slab。

第 1601~1602 行：把 slab 描述符存放在它该存放的地方。回想一下可以把 slab 描述符存放在 slab 自身或者第一个通用缓存中。

第 1605~1613 行：需要把页面与缓存和 slab 描述符关联。

第 1615 行：初始化 slab 中的所有对象。

```
-----
mm/slab.c
1616  if (local_flags & __GFP_WAIT)
1617      local_irq_disable();
1618  check_irq_off();
1619  spin_lock(&cachep->spinlock);
1620
1621  /* Make slab active. */
1622  list_add_tail(&slabp->list, &(list3_data(cachep)->slabs_free));
1623  STATS_INC_GROWN(cachep);
1624  list3_data(cachep)->free_objects += cachep->num;
1625  spin_unlock(&cachep->spinlock);
1626  return 1;
1627 opps1:
1628  kmem_freepages(cachep, objp);
1629 failed:
1630  if (local_flags & __GFP_WAIT)
1631      local_irq_disable();
1632  return 0;
1633  }
```

第 1616~1619 行：由于我们打算访问并且改变描述符字段，所以需要禁用中断并锁定数据。

第 1622~1624 行：添加新的 slab 描述符到缓存描述符的 `lists.slabs_free` 字段中。更新记录缓存大小的统计信息。

第 1625~1626 行：解开自旋锁并且成功返回。

第 1627~1628 行：该代码段将在页请求出错时被调用，基本上是释放请求的页。

第 1629~1632 行：启用中断，这样就允许中断传出了。

4.5.4 Slab 的销毁：退还内存与 `kmem_cache_destroy()`

缓存与 slab 都可被销毁。缓存可被缩减或者销毁以返还其占用的内存给空闲内存池。内核在系统内存过低时会调用析构函数。在任意一种情况下，slab 也可被销毁且将与之对应的页面返还给伙伴系统，以便重复利用。函数 `kmem_cache_destroy()` 用来销毁缓存。下面我们会详细分析该函数。函数 `kmem_cache_reap()` (`mm/slab.c`) 和 `kmem_cache_shrink` (`mm/slab.c`) 分别回收和缩减缓存。其中与伙伴系统的交互是通过函数 `kmem_freepages()` (`mm/slab.c`) 完成的。

`kmem_cache_destroy()`

如果要删除缓存，有几个实例可以使用。动态加载模块（假定载入和卸载过程中没有固定的永久内存预留）创建的缓存就必须在卸载时被销毁，从而可以释放内存，确保在下次模块载入时不出现重复缓存。因此，通常以这种方式销毁专用缓存。

销毁缓存的步骤与创建缓存的步骤相逆。在销毁缓存时不用关心对齐问题，只需要删除缓存描述符并释放内存即可。销毁缓存步骤可概括如下：

- (1) 从缓存链表中删除缓存；
- (2) 删除 slab 描述符；
- (3) 删除缓存描述符。

```
-----
mm/slab.c
1421 int kmem_cache_destroy (kmem_cache_t * cachep)
1422 {
1423     int i;
1424
1425     if (!cachep || in_interrupt())
1426         BUG();
1427
1428     /* Find the cache in the chain of caches. */
1429     down(&cache_chain_sem);
1430     /*
1431      * the chain is never empty, cache_cache is never destroyed
1432      */
1433     list_del(&cachep->next);
1434     up(&cache_chain_sem);
1435
1436     if (__cache_shrink(cachep)) {
1437         slab_error(cachep, "Can't free all objects");
1438         down(&cache_chain_sem);
1439         list_add(&cachep->next, &cache_chain);
1440         up(&cache_chain_sem);
1441         return 1;
1442     }
1443
```

```

...
1450  kmem_cache_free(&cache_cache, cachep);
1451
1452  return 0;
1453  }

```

该函数的参数 `cache` 是一个指针，指向待销毁缓存的描述符。

第 1425~1426 行：该代码段完成完整性检查，其中包括确认程序当前不在中断上下文中，并且缓存描述符不为空。

第 1429~1434 行：获得 `cache_chain` 信号量，从缓存链中删除指定缓存，释放 `cache_chain` 信号量。

第 1436~1442 行：该代码段负责释放未使用的 slab。若函数 `_cache_shrink()` 返回真，则表明缓存中仍然存在有 slabs，因此不可销毁缓存。这样要求我们按相反的顺序执行前面的步骤，重新将缓存描述符插入 `cache_chain` 链表中，当然同样需要先获取 `cache_chain` 信号量，操作完成后再释放它。

第 1450 行：最后，释放缓存描述符，结束操作。

4.6 内存请求路径

到目前为止，虽然我们已经讨论了 slab 分配器，但还未与任何实际的内存请求结合起来。除了缓存初始化函数外，我们还没有说明如何把这些函数配合在一起调用。因此，从现在开始我们来跟踪有关的内存请求的控制流程。当内核必须获得字节大小分组的内存块时，就需要使用函数 `kmalloc()`——它实际上会调用到函数 `kmem_getpages` 完成实际分配。调用路径如下所示：

```
kmalloc()->__cache_alloc()->kmem_cache_grow()->kmem_getpages()
```

4.6.1 `kmalloc()`

`kmalloc()` 函数在内核中分配内存对象：

```

-----
mm/slab.c
2098  void * __kmalloc (size_t size, int flags)
2099  {
2100      struct cache_sizes *csizes = malloc_sizes;
2101
2102      for (; csizes->cs_size; csizes++) {
2103          if (size > csizes->cs_size)
2104              continue;
2105      }
2106      ...
2112      return __cache_alloc(flags & GFP_DMA ?
2113          csizes->cs_dmacachep : csizes->cs_cachep, flags);
2114  }
2115  return NULL;
2116  }
-----

```

1. size

size 是请求的字节数。

2. flags

flags 指定内存请求的类型。这些标志将被传递给伙伴系统而不会影响 `kmalloc()` 的行为。

表 4-6 对这些标志进行了描述，有关它们的细节请参见 4.3.3 节。

表4-6 `vm_area_struct->vm_flags`的值

标 志	描 述
VM_READ	表示该区域中的页可读
VM_WRITE	表示该区域中的页可写
VM_EXEC	表示该区域中的页可执行
VM_SHARED	表示与其他进程共享该区域中的页
VM_GROWSDOWN	表示线性地址向低端内存扩展
VM_GROWSUP	表示线性地址向高端内存扩展
VM_DENYWRITE	表示该区域的页不可写
VM_EXECUTABLE	表示该区域内包含可执行代码
VM_LOCKED	表示页面被锁定
VM_DONTCOPY	表示页面不可被克隆
VM_DNTEXPAND	表示不可扩展该虚拟内存区

第 2102~2104 行：在查找缓存的过程中，找到所包含的对象大小大于所需大小的第一个缓存。

第 2112~2113 行：从 flags 参数指定的内存管理区中分配对象。

4.6.2 `kmem_cache_alloc()`

该函数是对函数 `__cache_alloc()` 的包装。它实际不实现任何额外的功能，其参数按如下方式传递给函数：

```
-----
mm/slab.c
2070 void * kmem_cache_alloc (kmem_cache_t *cachep, int flags)
2071 {
2072     return __cache_alloc(cachep, flags);
2073 }
-----
```

1. cachep

cachep 参数是我们的待分配对象的缓存描述符。

2. flags

flags 表示内存请求方式，它是按照函数 `kmalloc()` 的要求直接传递过来的。

内核提供的函数 `kfree()` 接口用于释放由 `kmalloc` 分配的字节大小的内存块，它接收的参数为指向由 `kmalloc()` 执行完毕所返回的内存的指针。图 4-9 描述了由 `kfree` 到 `kmem_freepages` 的调用过程。

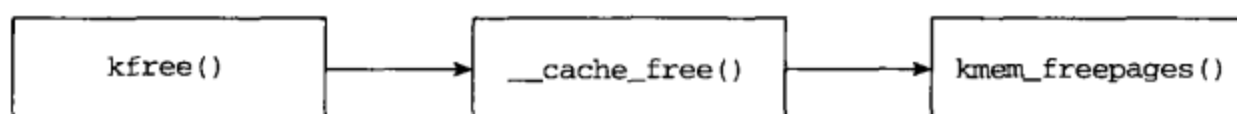


图 4-9 kfree()的调用图

4.7 Linux 进程的内存结构

到现在为止，我们已经讨论了内核如何管理自己的内存。现在要将注意力转向用户空间程序，观察内核如何管理它们使用的内存。在虚拟内存的帮助下，用户空间进程可有效地操作内存，就好像它可以访问系统中所有内存一样。事实上，内核会负责管理载入什么，如何载入以及何时进行这些操作。本节所讨论的内容就是内核如何管理内存，以及如何能对用户空间程序完全透明。

用户进程创建后便要分配一个虚拟地址空间。正如我们以前所讲的，进程的虚拟地址空间是一段未分段的、进程可访问的线性地址范围。这段地址的大小取决于系统体系结构中寄存器的大小。大多数系统有 32 位的地址空间，有些系统（如 A G5）的处理器具有 64 位的地址空间。

进程被创建后，其地址范围可以通过增加或删除线性地址间隔（address interval）得以扩大或缩减。地址间隔（一段地址区间）是一种内存单元，它也被称为内存范围（memory region）或内存区（memory area）。把进程地址空间划分成不同类型的区域是有用的，不同的内存区具有不同的保护方案和特点。进程内存区的保护方案与进程上下文相关，比如程序代码段的某些部分标记为只读（text），而其他部分则标记为可写（变量）或可执行（指令）。同样，一个特殊进程可能仅能访问属于它的内存区。

在内核中，进程地址空间以及与之相关的所有信息都保存在 `mm_struct` 描述符中。回想一下第 3 章，该结构出现在进程控制结构 `task_struct` 中。一个内存区由 `vm_area_struct` 描述符表示。每个内存区描述符都描述它所表示的一段连续地址区间。本章中，我们将称地址区间描述符为内存区间描述符或 `vma_area_struct`。接下来看看 `mm_struct` 和 `vm_area_struct` 结构。图 4-10 描述了这些数据结构之间的关系。

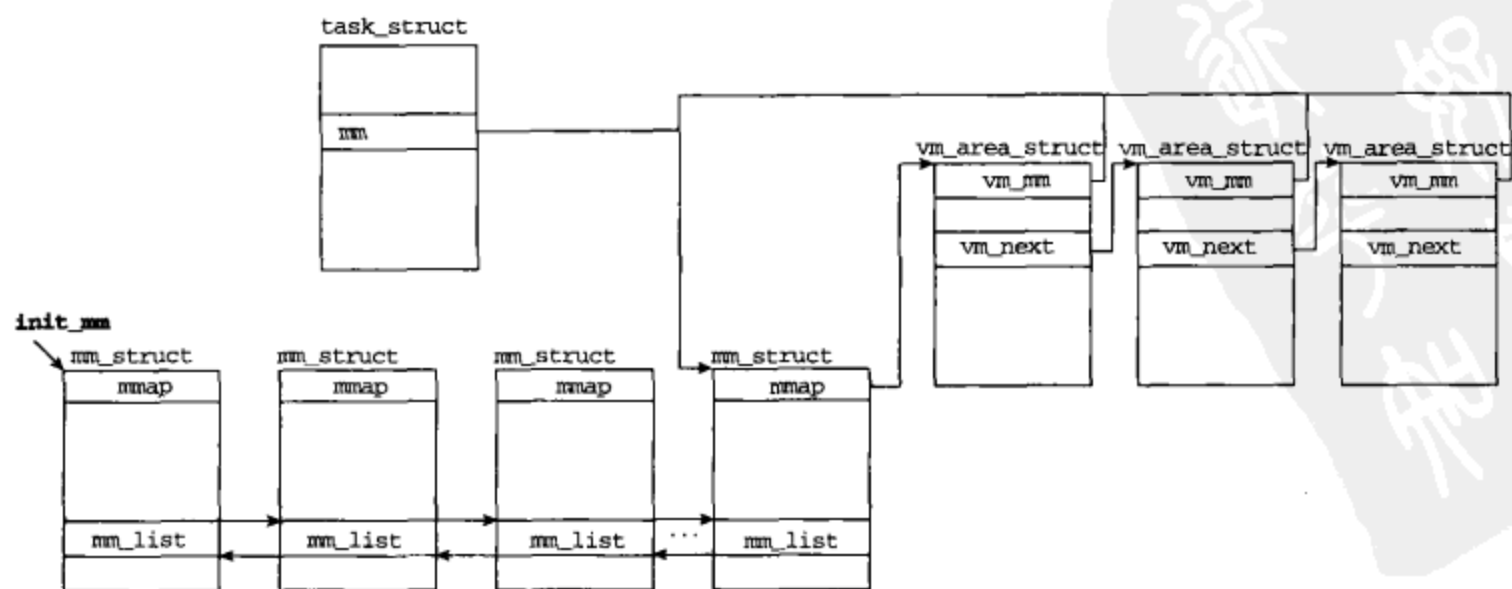


图 4-10 进程相关的内存结构体

4.7.1 mm_struct

每一个任务都有一个 mm_struct (include/linux/sched.h) 结构, 内核用它来表示它自己的内存地址范围。所有 mm_struct 描述符统一存放在一个双向链表中。链表头是对应于进程 0 的 mm_struct, 进程 0 是空闲进程, 可通过全局变量 init_mm 来访问该描述符。

```
-----
include/linux/sched.h
185 struct mm_struct {
186     struct vm_area_struct * mmap;
187     struct rb_root mm_rb;
188     struct vm_area_struct * mmap_cache;
189     unsigned long free_area_cache;
190     pgd_t * pgd;
191     atomic_t mm_users;
192     atomic_t mm_count;
193     int map_count;
194     struct rw_semaphore mmap_sem;
195     spinlock_t page_table_lock
196
197     struct list_head mmlist;
198     ...
202     unsigned long start_code, end_code, start_data, end_data;
203     unsigned long start_brk, brk, start_stack;
204     unsigned long arg_start, arg_end, env_start, env_end;
205     unsigned long rss, total_vm, locked_vm;
206     unsigned long def_flags;
207     cpumask_t cpu_vm_mask;
208     unsigned long swap_address;
209     ...
228 };
-----
```

1. mmap

进程所对应的内存区描述符 (下一节给出定义) 被分配给链接于一个链表中的进程。通过 mm_struct 的 mmap 字段访问该链表, 而 vma_area_struct 中的 vm_next 字段将各个内存区描述符结点链接起来。

2. mm_rb

虽然用简单链表链接内存描述符是检索指定进程对应的内存区描述符最方便、简单的方法。但是这样搜索一个指定的内存描述符, 简单链表是无法提供高性能检索的。所以进程地址空间对应的内存描述符是以红黑树的形式存放的, mm_rb 字段便指向树根节点。利用红黑数存放结点确保了内核能以最快速度访问指定的内存区描述符。

3. mmap_cache

mmap_cache 是指向进程最后一个内存区描述符的指针。由于内存地址访问的局部性原理 (principle of locality), 临近于最后访问地址的内存位置最有可能接下来被很快访问到。于是, 很有可能当前检查的地址和最后一次检查地址是在同一个内存区。要知道当前地址和最后一次访问地址处于同一个内存区的可能性约为 35%。

4. pgd

Pgd 是一个指针, 指向存放该内存区中页的页全局目录。在空进程(进程 0) 对应的 mm_struct 中, 该字段指向 swapper_pg_dir 目录。请参见 4.9 节了解该字段的更多信息。

5. mm_users

mm_users 字段存放访问该内存区的进程数量。轻量进程或者线程共享同一地址区间和内存区, 所以线程对应的 mm_struct 其 mm_users 字段值通常大于 1。通过以下原子函数对该字段进行操作: atomic_set()、atomic_dec_and_lock()、atomic_read() 和 atomic_inc()。

6. mm_count

mm_count 字段是对 mm_struct 结构的使用统计。当确定 mm_struct 结构是否可以被释放时, 就需要检查该字段。如果其值为 0, 则说明没有进程正在使用它, 因此可以将其回收。

7. map_count

map_count 字段存放在进程地址空间中的内存区数量, 或者说 vma_area_struct 描述符的数量。每当一个新内存区加入到进程地址空间时, 随着 vma_area_struct 结构插入 mmap 链表和 mm_rb 树中, 该字段的值加 1。

8. mm_list

类型为 list_head 的 mm_list 字段存放内存描述符链表中相邻的 mm_structs 结构的地址。如前面所说链表头由全局变量 init_mm 指向, 它是进程 0 的内存描述符。当操作该链表时, 需要 mm_list_lock 锁, 保护其不被并发访问。

接下来讨论的 11 个字段用于处理进程需要分配给它们的各种内存区。为了避免偏离讨论进程内存相关结构这一主题, 我们只对这些字段作粗略的描述。

9. start_code 和 end_code

start_code 和 end_code 字段存放进程在内存中代码段 (也就是可执行的正文段) 的开始地址和结束地址。

10. start_data 和 end_data

start_data 和 end_data 字段存放初始化数据 (可执行文件中 .data 部分) 的起始地址和结束地址。

11. start_brk 和 brk

start_brk 和 brk 字段存放进程堆的开始和结束地址。

12. start_stack

start_stack 存放进程栈的开始地址。

13. arg_start 和 arg_end

arg_start 和 arg_end 存放传递给进程的参数的起始和结束地址。

14. env_start 和 env_end

env_start 和 env_end 字段存放环境段的开始和结束地址。

本章主要讨论了 mm_struct 中的各个字段的含义。接下来我们来看看内存区描述符 vm_area_struct 中的各个字段。

4.7.2 vm_area_struct

vm_area_struct 结构定义了虚拟内存区。进程具有各种不同的内存区，但是每个内存区都只对应一个 vm_area_struct 结构。

```
-----
include/linux/mm.h
51 struct vm_area_struct {
52     struct mm_struct * vm_mm;
53     unsigned long vm_start;
54     unsigned long vm_end;
...
57     struct vm_area_struct *vm_next;
...
60     unsigned long vm_flags;
61
62     struct rb_node vm_rb;
...
72     struct vm_operations_struct * vm_ops;
...
};
-----
```

1. vm_mm

所有的内存区都属于进程的地址空间，而地址空间由 mm_struct 表示。结构 vm_mm 便指向该内存区所在的地址空间的 mm_struct 结构。

2. vm_start和vm_end

内存区都和一个地址区间关联。在 vm_area_struct 结构中每个地址区间都有自己的起始和结束位置。考虑到性能问题，内存区的开始地址必须是页面大小的整数倍，而且内核确保用指定内存区中的数据填充页面时也是按照页面大小的整数倍进行。

3. vm_next

vm_next 指向链表中下一个 vm_area_struct 结构，链表包含进程地址空间所有的内存区。链表头是由进程地址空间对应的 mm_struct 结构中 mmap 字段所指向的。

4. vm_flags

内存区间具有自己相关的特性，这些特性都存放在 vm_flags 字段中，并适用于内存区中的页。表4-6描述了这些标志的可能取值。

5. vm_rb

vm_rb 存放该内存区对应的红黑树节点。

6. vm_ops

vm_ops 字段是由函数指针组成的结构体，这些函数用于操作特定的 vm_area_struct，包括打开内存区、关闭内存区、反映射内存区等，另外还包含一个函数指针，缺页异常发生时调用该函数。

4.8 进程映像的分布及线性地址空间

当用户空间程序被载入内存后，它就拥有自己的线性地址空间，该空间被划分成各种内存区或叫做段 (segment)。按照进程执行功能上的差异，线性空间被划分成一个个段。这些功能上独立的段被映射到进程地址空间中。与进程执行相关的主要段有 6 个。

- **text**: 该段也被称作代码段，它存放程序的可执行指令。因此它具有 `execute` 和 `read` 属性。如果同一程序被实例化为多个进程，则同一指令载入两次必然造成内存浪费，因此 Linux 允许多进程共享 **text** 段。`mm_struct` 中的 `start_code` 和 `end_code` 字段保存了 **text** 段的起始位置和结束位置。
- **Data**: 该段存放所有已初始化的数据。初始化数据包含静态分配的数据和初始化的全局数据。下面的代码段给出了初始化数据的示例。

```
-----  
example1.c  
int gvar = 10;  
  
int main(){  
...  
}
```

- **gvar**: 全局变量，它被初始化后存放在数据段中。该段具有可读、可写属性，但是不可被运行在同一程序中的其他进程共享。`mm_struct` 结构的 `start_data` 和 `end_data` 字段存放数据段的起始地址和结束地址。
- **BSS**: 该段存放未初始化数据。这些数据包括程序执行时被系统初始化成 0 的全局变量。该段的另一个名字叫做初始化为 0 的数据段。下面的代码段给出了未初始化的数据示例：

```
-----  
example2.c  
int gvar1[10];  
long gvar2;  
  
int main() {  
...  
}
```

该段内的对象仅仅有名称和大小属性。

- **堆 (heap)**, 该段用于扩展进程的线性地址空间。当程序调用函数 `malloc()` 获取动态内存时，新获取的动态内存便被置于堆中。`mm_struct` 结构的 `start_brk` 和 `brk` 字段保存堆的起始地址和结束地址。当 `malloc` 被调用获得动态内存时，系统调用 `sys_brk()` 将 `brk` 指针移动到新位置，从而扩展了堆空间。
- **栈 (stack)**, 该段包含所有已分配内存的局部变量。当函数被调用时，函数的局部变量被压入栈。当函数结束时，与函数相关的变量被弹出栈。其他信息，包括返回地址和参数同样也被压入栈。`mm_struct` 结构中的 `start_stack` 字段标记了进程栈的起始位置。

虽然进程执行与上述 6 个主要段相关，但它们只映射到地址空间中的 3 个内存区。这些内存区分别叫做 text data 和 stack。data 段包括可执行程序的初始化 data 段、bss 和堆，text 段包含可执行程序的 text 段。图 4-11 描述了线性地址空间的结构以及 mm_struct 如何跟踪记录这些段。

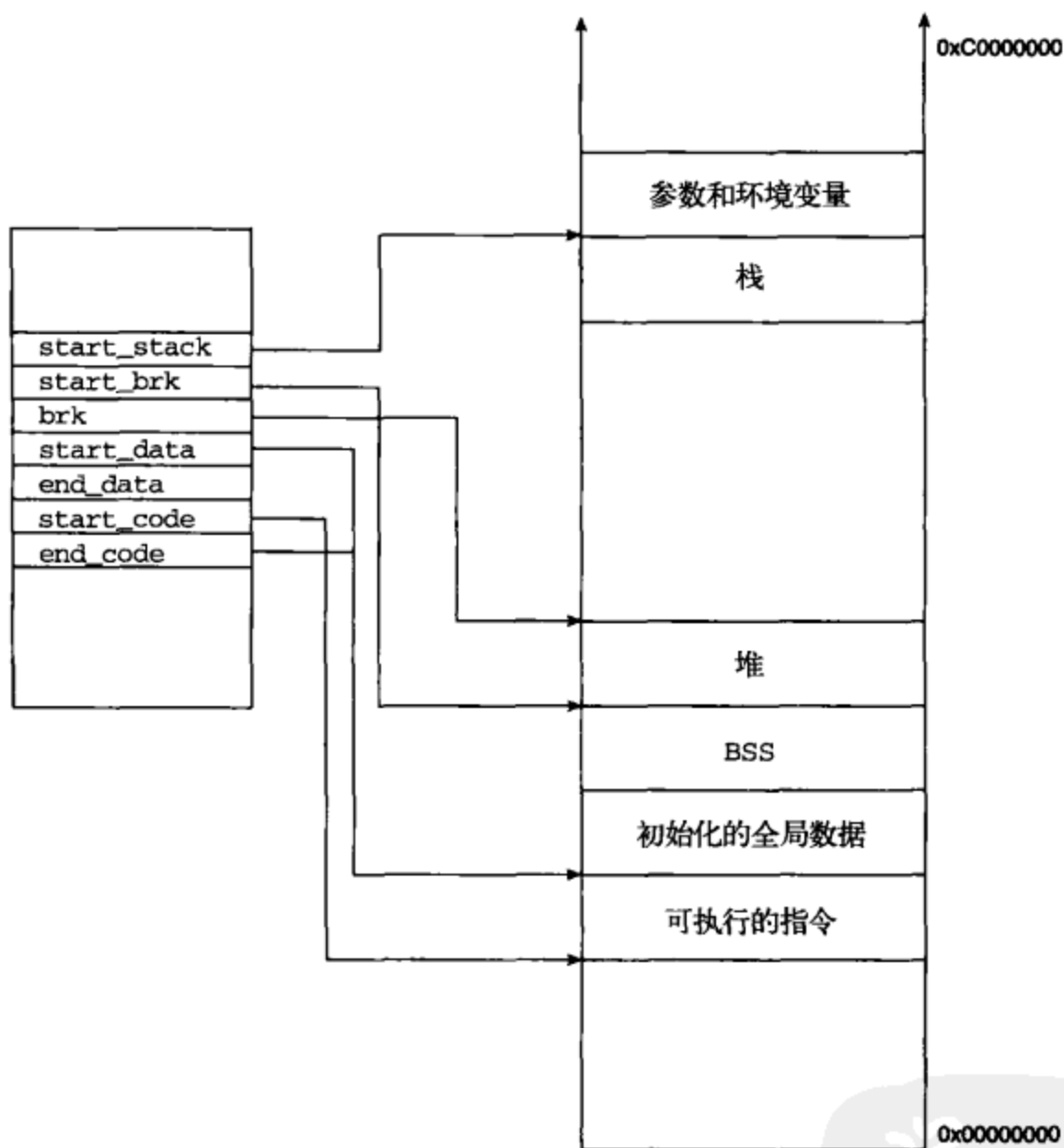


图 4-11 进程地址空间

各种内存区都被映射到 /proc 文件系统中，可通过 /proc/<pid>/maps 的输出访问进程的内存映射。我们通过一个例子来看看进程地址空间中的所有内存区。example3.c 中的代码为我们演示了被映射的程序。

```
-----
example3.c
#include <stdio.h>
int main(){
    while(1);
    return(0);
}
-----
```


图 4-12 描述了示例程序执行后输出的 `/proc/<pid>/maps` 中的内容。

```

08048000-08049000 r-xp 00000000 03:05 1324039 /home/1kp/example3
08049000-0804a000 rw-p 00000000 03:05 1324039 /home/1kp/example3
40000000-40015000 r-xp 00000000 03:05 767058 /lib/ld-2.3.2.so
40015000-40016000 rw-p 00014000 03:05 767058 /lib/ld-2.3.2.so
40016000-40017000 rw-p 00000000 00:00 0
42000000-4212e000 r-xp 00000000 03:05 1011853 /lib/tls/libc-2.3.2.so
4212e000-42131000 rw-p 0012e000 03:05 1011853 /lib/tls/libc-2.3.2.so
42131000-42133000 rw-p 00000000 00:00 0
bffffe00-c0000000 rwxp fffff000 00:00 0

```

图 4-12 `cat/proc/<pid>/maps`

最左边的列显示的是内存段的范围，也就是段的起始地址和结束地址。下一列显示的是段的访问权限。这些权限标识类似于文件访问权限：`r` 表示可读、`w` 表示可写、`x` 表示可执行，最后一个权限可能是 `p` 表示私有段，或者 `s` 表示共享段（一个私有段并非必然是不可共享的）。`p` 表示仅当前不可共享。下一列表示段的偏移量。左起第 4 列为由冒号分割的两个数字，它们分别代表与段相关的文件在文件系统中的主设备号从设备号（有些段不存在对应文件，所以该列值为 `00:00`）。第 5 列表示文件对应的索引节点，第 6 列和最右边的列存放文件名。如果段没有对应的文件名，那么该列为空，而且索引节点列为 0。

在我们的例子中，第一行是实例程序中代码段的描述。可以看到它的访问权限为可执行，下一行描述了实例程序的数据段，注意它的访问权限为可写。

我们实例程序被动态链接，这意味着它用到的库函数在运行时才被加载。这些库函数也需要被映射到进程地址空间才能访问。下面 6 行便描述了动态链接库的段。开始 3 行依次为 `ld` 库的代码段、数据段和 `bss` 段。在下面三行则依次是 `libc` 的代码段、数据段和 `bss` 段。

最后一行，其权限表示它可读、可写、可执行，它代表的是进程栈空间，最高可扩展到 `0xC0000000`，`0xC0000000` 是用户空间进程可访问的最高内存地址。

4.9 页表

程序内存的有效管理是通过虚拟地址完成的。但现在面临的问题是，当把指令提交给处理器时，处理器却对虚拟地址无能为力。处理器只能操作物理地址。虚拟地址和对应的物理地址之间的管理便要靠内核（借助于硬件）中的页表来维护。

页表对内存中的页面走向进行记录。在内核运行的整个生命期中，页表都存放在内存中。Linux 有称为 3 级页表的分页机制。3 级页表确保 64 位体系结构有足够的空间维护它们的虚拟地址到物理地址的转换。从名字我们可以看出，3 级页表结构有三种不同的页表：顶层目录称作 PGD (Page Global Directory)，它由数据类型 `pdg_t` 表示；第 2 级页表称为 PMD (Page Middle Directory)，它由数据类型 `pmd_t` 表示；最后一级页表称为 PTE (Page Table)，它由数据类型 `pte_t` 表示。图 4-13 描述页表结构。

PGD 存放的项指向 PMD，PMD 存放的项指向 PTE，而 PTE 存放的项指向特定页面。每个进程都有自己的页表。`mm_struct->pgd` 字段指向进程的 PGD。32 位或 64 位的虚拟地址被分成各

种大小（取决于体系结构）的偏移字段。这些字段对应于 PGD、PMD、PTE 和页面本身的偏移。

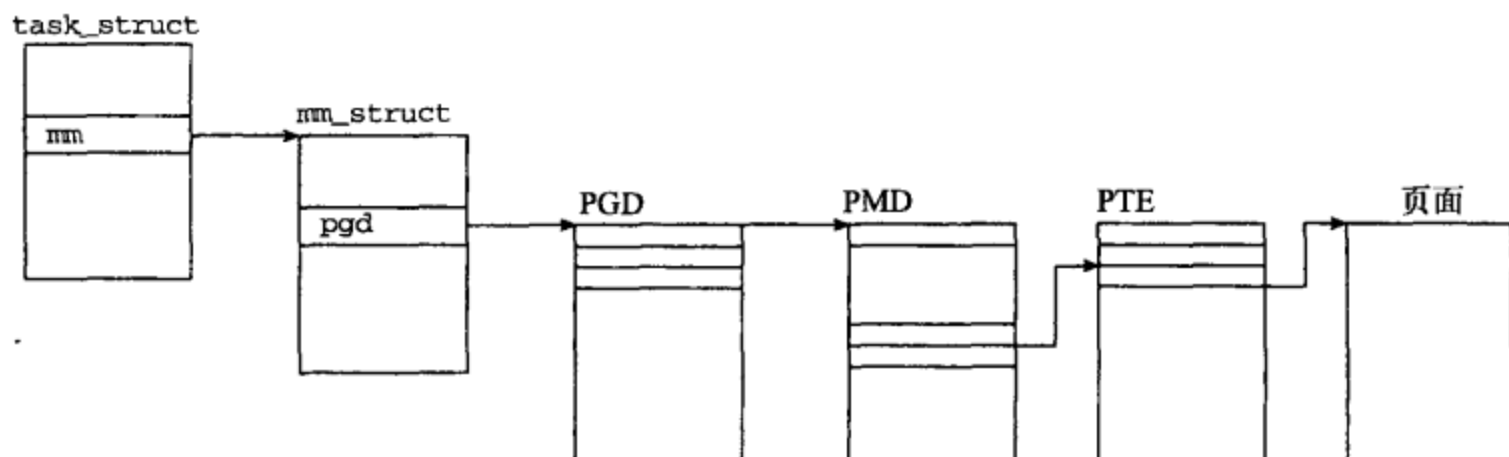


图 4-13 Linux 的页表结构

4.10 缺页

在进程运行期间，很有可能发生所访问的地址在进程地址空间，但并不在内存的情况；也有可能要访问的页在内存，但访问行为与页设置不符（比如对只读区域进行写操作）。当上述情况发生时，系统就产生缺页（page fault）。缺页是一种异常处理程序，它管理在程序页访问期间所发生的错误。当硬件发起（内核捕获的）缺页异常时，页将被从存储设备取回，内核就分配缺失的页。

每种体系结构都有一个与体系结构相关的函数处理缺页。x86 和 PPC 都调用函数 `do_page_fault()`。x86 的缺页处理函数 `do_page_fault(*regs, error_code)` 位于文件 `/arch/i386/mm/fault.c` 中；PowerPC 缺页处理函数 `do_page_fault(*regs, address, error_code)` 位于 `/arch/ppc/mm/fault.c` 文件中。这两个函数的功能非常相似，所以我们只需要讨论 x86 的 `do_page_fault()` 函数，就可了解 PowerPC 版本函数的功能。

两种体系结构处理缺页的不同主要在于调用函数 `do_page_fault()` 前，错误信息如何获取和保存。我们首先解释 x86 缺页处理的特点，接着将解释 `do_page_fault()` 函数。结束时我们将解释它与 PowerPC 的区别。

4.10.1 x86 缺页异常

x86 的缺页处理函数 `do_page_fault()` 在硬件中断 14 发生时被调用。在处理器发现下列条件为真时该中断发生。

- (1) 允许请页，该地址的页目录或页表项的存在位为 0。
 - (2) 允许请页，当前特权级（特权级别数字低者特权高）低于访问请求页所需的特权。
- 一旦缺页中断发生，处理器保存两个信息。

- (1) 错误的信息被压栈，错误信息存放在字的低 4 位（`do_page_fault()` 函数未使用第 3 位）。

参见表 4-7 了解各位值的对应含义。

表4-7 缺页error_code

	位 2	位 1	位 0
Value = 0	内核	读	页不存在
Value = 1	用户	写	保护错误

(2) 引起异常的 32 位线性地址，该地址存放在寄存器 cr2 中。

函数 do_page_fault() 的参数 regs 是一个结构体，其中包含的是系统寄存器，error_code 参数使用其中低 3 位描述缺页的原因。

4.10.2 缺页处理程序

在两种体系结构中，do_page_fault() 函数都使用给定信息，且执行几种特定操作之一。这些代码段按照相当复杂的流程检查缺页，并按下列几种方式之一完成操作。

- 由 handle_mm_fault() 函数找到缺页发生的偏移地址。
- 著名的 oops 转储（在标记 no_context: 处），Powerpc 的 bad_page_fault()。
- 产生一个段错误，（在标记 bad_area: 处），PowerPc 调用 bad_page_fault() 处理。
- 返回错误给调用者（修正）。

```
-----
arch/i386/mm/fault.c
212  asmlinkage void do_page_fault(struct pt_regs *regs, unsigned long error_code)
213  {
214      struct task_struct *tsk;
215      struct mm_struct *mm;
216      struct vm_area_struct * vma;
217      unsigned long address;
218      unsigned long page;
219      int write;
220      siginfo_t info;
221
222      /* get the address */
223      __asm__("movl %%cr2,%0":"=r" (address));
224      ...
232      tsk = current;
233
234      info.si_code = SEGV_MAPERR;
-----
```

第 223 行：发生缺页的地址被存放在 cr2 控制寄存器中，该线性地址被读入，并设置本地变量 address 来保存该值。

第 232 行：把 task_struct 类型指针 tsk 设置为指向当前任务的 task_struct 结构。

现在，我们可以找到缺页产生的地址，图 4-14 描述下列代码段的执行流程。

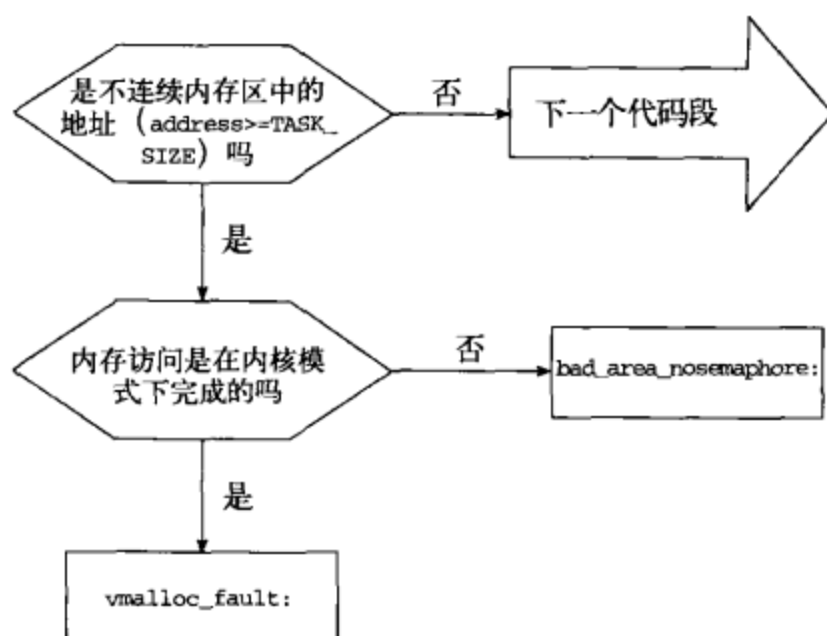


图 4-14 缺页 I

```

-----
arch/i386/mm/fault.c
246  if (unlikely(address >= TASK_SIZE)) {
247      if (!(error_code & 5))
248          goto vmalloc_fault;
...
253  goto bad_area_nosemaphore;
254  }
...
257  mm = tsk->mm
...
-----

```

第 246~248 行：该代码段检查发生缺页的地址是否位于内核模块的空间（也就是说处于非连续内存区中）。非连续内存区地址的线性地址 $\geq \text{TASK_SIZE}$ 。如果确实发生在非连续区，则继续检查 `error_code` 的 0 位和 2 位是否为空。回想表 4-7 可知，该错误是由于访问不存在的内核页面造成的。如果真是这样，则说明缺页发生在内核态，于是标记 `vmalloc_fault:` 处的代码将被调用。

第 253 行：如果我们执行到这一行，则意味着虽然访问发生在非连续内存区，但它仍然发生在用户态，或触发了保护错误，或两者都有。在这种情况下，跳到标记 `bad_area_nosemaphore:` 处执行。

第 257 行：该行代码设置局部变量 `mm` 指向当前任务的内存描述符，如果当前任务是内核线程，那么该值为 `NULL`，这点在下面的代码中变得很有意义。

此刻，我们已经确定了缺页不是发生在非连续内存区，图 4-15 描述下面代码的执行流程。

```

-----
arch/i386/mm/fault.c
...
262  if (in_atomic() || !mm)
263      goto bad_area_nosemaphore;

```

```

264
265 down_read(&mm->mmap_sem);
266
267 vma = find_vma(mm, address);
268 if (!vma)
269     goto bad_area;
270 if (vma->vm_start <= address)
271     goto good_area;
272 if (!(vma->vm_flags & VM_GROWSDOWN))
273     goto bad_area;
274 if (error_code & 4) {
...
281     if (address + 32 < regs->esp)
282         goto bad_area;
283 }
284 if (expand_stack(vma, address))
285     goto bad_area;
...

```

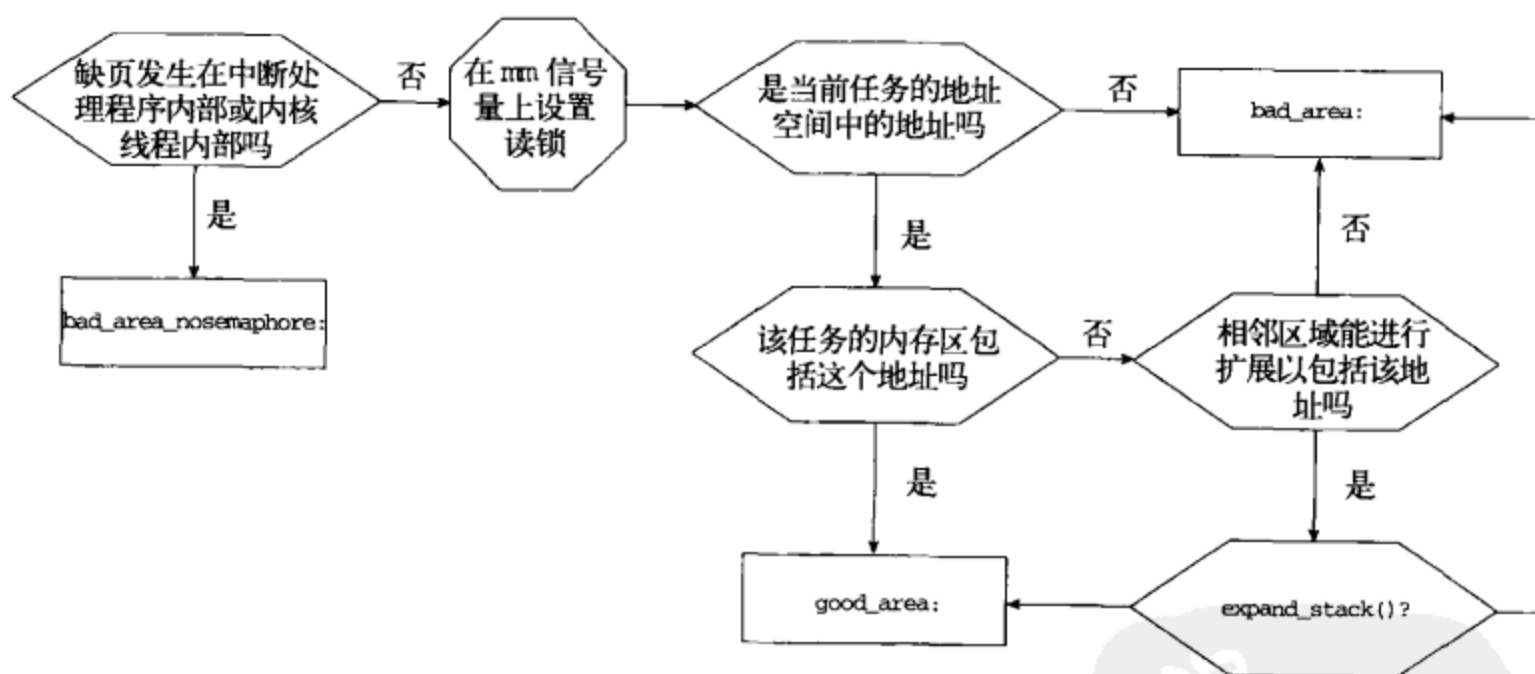


图 4-15 缺页 II

第 262~263 行：该代码段检查缺页是否发生于中断处理程序或内核空间中，如果是，那么我们转到标记 `bad_area_semaphore:` 处执行。

第 265 行：这里我们将准备搜索当前进程的内存区，所以我们在内存描述符的信号量上设置读锁。

第 267~269 行：在这里，假定我们知道缺页产生的地址不是在内核线程或中断处理程序中，于是就搜索进程地址空间查看这个地址是否处于进程的某个内存区，如果不在，则跳到标记 `bad_area:` 处执行。

第 270~271 行：如果在进程地址空间中找到了一个可用的内存区，则跳到标记 `good_area:` 处执行。

第 272~273 行：如果找到的内存区不可用，那么检查最临近的区间是否可扩展，以满足页请

求, 如果不可, 则跳到标记 `bad_area:` 处执行。

第 274~284 行: 否则, 出错地址可能由栈操作引起, 如果扩展栈没有什么帮助, 则跳到标记 `bad_area:` 处执行。

接下来, 继续解释跳转标记点处代码的意义。我们从标记 `vmalloc_fault` 处开始解释。如图 4-16 所示。

```
-----
arch/i386/mm/fault.c
473 vmalloc_fault:
{

    int index = pgd_index(address);
    pgd_t *pgd, *pgd_k;
    pmd_t *pmd, *pmd_k;
    pte_t *pte_k;

    asm("movl %%cr3,%0":"=r" (pgd));
    pgd = index + (pgd_t *)__va(pgd);
    pgd_k = init_mm.pgd + index;

491    if (!pgd_present(*pgd_k))
        goto no_context;

    pmd = pmd_offset(pgd, address);
    pmd_k = pmd_offset(pgd_k, address);
    if (!pmd_present(*pmd_k))
        goto no_context;
    set_pmd(pmd, *pmd_k);

    pte_k = pte_offset_kernel(pmd_k, address);
506    if (!pte_present(*pte_k))
507        goto no_context;
508    return;
509 }
```

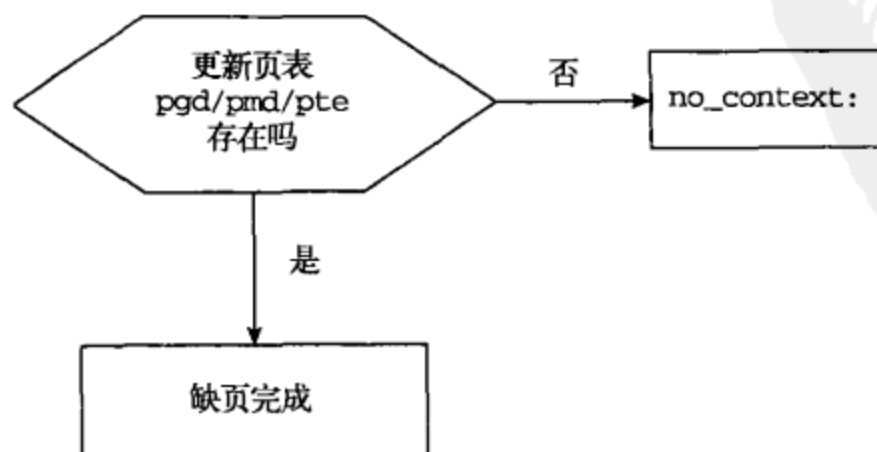


图 4-16 标记 `vmalloc_fault`

第 473~509 行: 当前进程的页全局目录被获取 (通过 `cr3` 寄存器) 并且存放在变量 `pgd` 中,

内核页全局目录被取得并放入变量 `pgd_k` (按同样的方式处理 `pmd` 和 `pte` 变量) 中。如果出错地址不在内核分页系统中, 则代码跳到标记 `no_context:` 处, 否则当前进程使用内核的 `pgd`。

现在, 我们看看标记 `good_area:`。此刻我们知道保存出错地址的内存区在进程的地址空间中, 接下来需要确认地址的访问权限是否正确。图 4-17 给出了流程图。

```
-----
arch/i386/mm/fault.c
290 good_area:
291     info.si_code = SEGV_ACCERR;
292     write = 0;
293     switch (error_code & 3) {
294         default: /* 3: write, present */
295     ...
        /* fall through */
300     case 2: /* write, not present */
301         if (!(vma->vm_flags & VM_WRITE))
302             goto bad_area;
303         write++;
304         break;
305     case 1: /* read, present */
306         goto bad_area;
307     case 0: /* read, not present */
308         if (!(vma->vm_flags & (VM_READ | VM_EXEC)))
309             goto bad_area;
310     }
```

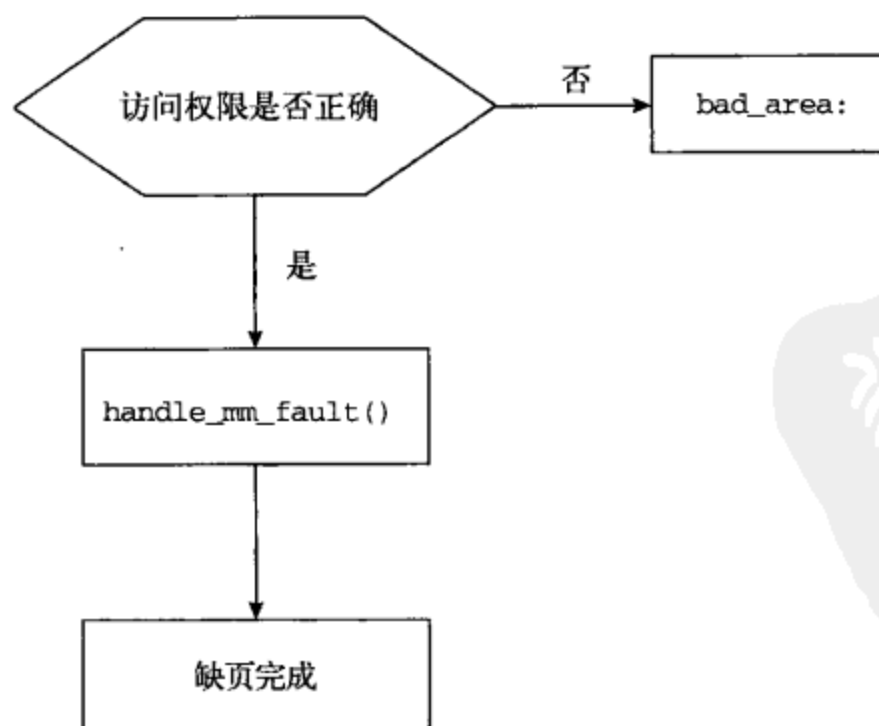


图 4-17 标记 `good_area`

第 294~304 行: 如果缺页是由一个内存读操作引起的 (回想如果是这种情况, 则 `error code` 最左边的位被设置为 1)。我们检查内存区是否是可写的, 如果是不可写的, 那么说明访问权限不匹配, 跳到标记 `bad_area:` 处执行; 如果是可写的, 则依次判断各种 `case` 情况, 并最终进入

函数 `handle_mm_fault()` 执行, 同时将局部变量 `write` 置为 1。

第 305~309 行: 如果缺页是由读或执行访问触发的, 而且页存在。那么我们跳转到标记 `bad_area:` 处执行, 因为显而易见访问权限冲突。如果页不存在, 则检查内存区是否具有可读或可执行属性。如果没有, 依然跳转到标记 `bad_area:` 处, 因为即使要取回页, 访问权限保护依然不允许访问; 如果有, 依次判断各种 `case` 情况, 最终进入函数 `handle_mm_fault()` 执行, 同时局部变量 `write` 被置 0。

下面标记处的代码是在权限检查通过后被执行的。它被标记为 `survive:`

```
-----
arch/i386/mm/fault.c
survive:
318  switch (handle_mm_fault(mm, vma, address, write)) {
    case VM_FAULT_MINOR:
        tsk->min_flt++;
        break;
    case VM_FAULT_MAJOR:
        tsk->maj_flt++;
        break;
    case VM_FAULT_SIGBUS:
        goto do_sigbus;
    case VM_FAULT_OOM:
        goto out_of_memory;
329  default:
        BUG();
    }
-----
```

第 318~329 行: 函数 `handle_mm_fault()` 被调用时, 所提供的参数有: 当前内存描述符 (`mm`)、出错地址所在的内存区描述符、出错地址, 以及访问是否可读、可执行或可写。switch 帮助我们捕捉错误, 确保正确地退出程序执行。

下面的代码片描述了标记 `bad_area` 和 `bad_area_no_semaphore`。当跳到该处执行时, 我们知道下列条件至少其中之一发生。

- (1) 产生缺页的地址不在进程地址空间, 因为已搜索了内存区, 并未发现有匹配的。
 - (2) 产生缺页的地址不在进程地址空间, 而且该地址所在的内存区不能扩展来存放它。
 - (3) 产生缺页的地址在进程地址空间, 但内存区的访问权限与希望执行的操作不匹配。
- 现在需要确定访问是否发生在内核态, 下面的代码和图 4-18 描述这些标记的处理流程。

```
-----
arch/i386/mm/fault.c
348  bad_area:
349  up_read(&mm->mmap_sem);
350
351  bad_area_nosemaphore:
352  /* User mode accesses just cause a SIGSEGV */
353  if (error_code & 4) {
354  if (is_prefetch(regs, address))
355  return;
356
```



```

357     tsk->thread.cr2 = address;
358     tsk->thread.error_code = error_code;
359     tsk->thread.trap_no = 14;
360     info.si_signo = SIGSEGV;
361     info.si_errno = 0;
362     /* info.si_code has been set above */
363     info.si_addr = (void *)address;
364     force_sig_info(SIGSEGV, &info, tsk);
365     return;
366 }

```

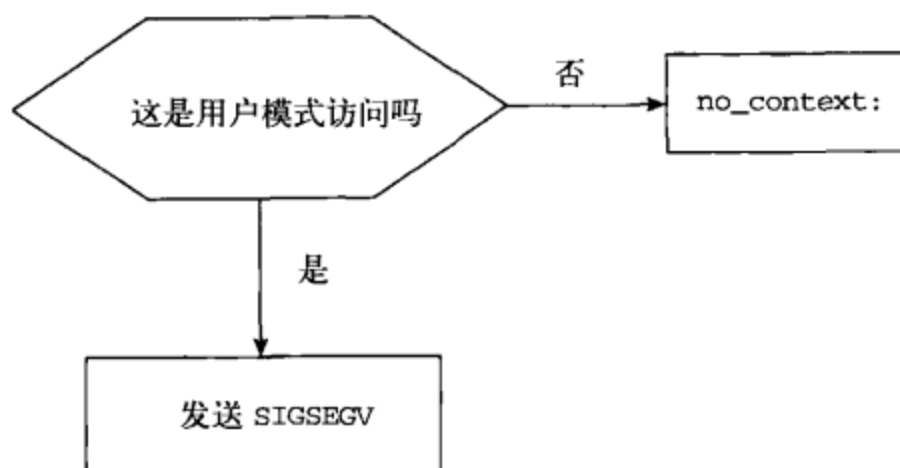


图 4-18 标记 bad_area

第 348 行：函数 `up_read()` 释放进程内存描述符信号量上的读锁。注意，当我们在内存描述符的信号量加上读锁，以遍历内存区，从而查看地址是否处于进程地址空间中之后，只跳到标记 `bad_area:` 处执行；否则跳入到标记 `bad_area_nosemaphore:` 处执行。两者不同之处在于对信号量上加读锁。

第 351~353 行：因为地址不在地址空间中，我们检查错误是否发生于用户态。回想表 4-7，错误代码值 4 表示错误发生于用户态。

第 354~366 行：我们已经确认缺页发生在用户态，所以要发送 `SIGSEGV` 信号（陷入 14）。

下面的代码段是标记 `no_context:` 的流程，当跳到这点执行时，我们知道下列条件之一发生了：

- ❑ 缺少某个页表；
- ❑ 在内核态中内存访问未完成。

图 4-19 描述了标记 `no_context` 代码段的流程。

```

arch/i386/mm/fault.c
388 no_context:

390     if (fixup_exception(regs))
        return;

432     die("Oops", regs, error_code);
        bust_spinlocks(0);

```

```
do_exit(SIGKILL);
```

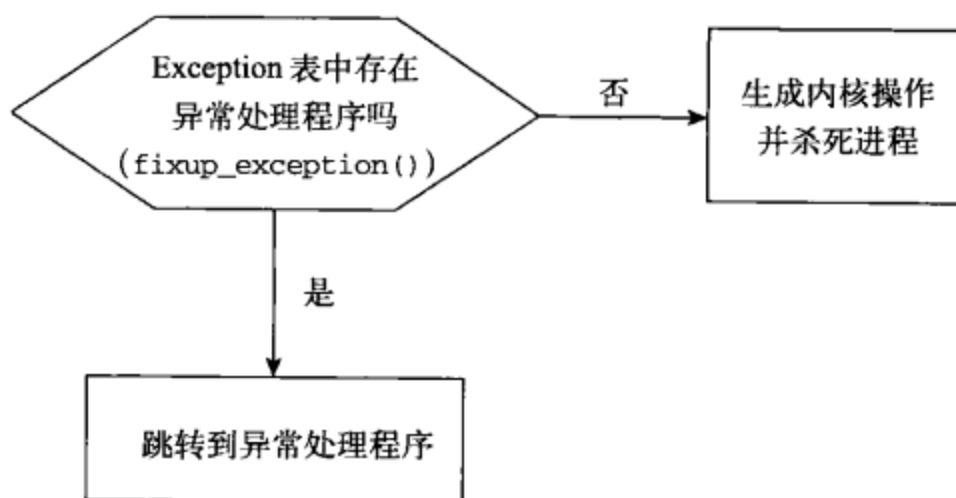


图 4-19 标记 no_context

第 390 行：函数 `fixup_exception()` 利用传入的 `eip` 参数在异常表中搜索出错的指令。如果指令存在于表中，那么说明该指令已经与内置“隐藏”的错误处理代码同被编译。缺页处理程序 `do_page_fault()` 使用错误处理代码作为返回地址，并且跳到该地址。该代码可标记一个错误。

第 432 行：如果没有在异常表中找到对应的出错指令，则跳到标记 `no_context:` 处执行，且代码以 `oops` 屏幕转储错误结束调用。

4.10.3 PowerPC 缺页异常

PowerPC 缺页处理函数 `do_page_fault()` 在指令或数据存放发生异常时被调用。因为在 PowerPC 处理器中各种版本存在细微差别，错误码有一些细微的格式不同，但是记录的信息都类似，表示产生错误的原因是读还是写，或是保护错误。PowerPC 的缺页处理程序 `do_page_fault()` 不会产生 `oops` 错误。

在 PowerPC 中，标记 `no_context` 处代码与标记 `bad_area` 处代码合并在一起，并且被置于 `bad_page_fault()` 函数中。该函数以产生段错误结束，而且该函数同样利用 `fixup` 函数检索异常表 `exception_table`。

4.11 小结

本章不但介绍与内存管理有关的所有概念，还解释了各个概念的实现。第一个概念是页，它是内核进行内存管理的基本单元。我们介绍了如何在内核中跟踪页，然后讨论了内存管理区，内存管理区受制于硬件限制，是对内存的一种划分。另外还讨论了 Linux 使用的页面分配和回收算法，这就是所谓的伙伴系统。

在讨论了页和页面管理的基本知识后，接着讨论了小于一页的小块内存分配，即由 slab 分配器管理的内存。要分配小内存块就需要借助函数 `kmalloc()` 和相关内核内存分配函数。我们跟踪了这些函数的执行过程，观察它们如何与 slab 分配器交互。到此为止，内核内存管理结构讨论完毕。